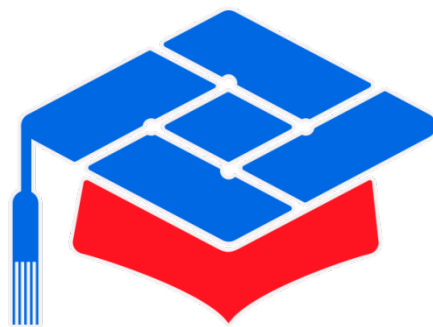




第九届

全国大学生集成电路创新创业大赛

报告类型*: 设计报告

参赛杯赛*: 紫光同创杯

作品名称*: 基于 FPGA 的双机械臂协同控制系统设计

队伍编号*: CICC0900959

团队名称*: 我来带队

目录

1 作品介绍.....	1
1.1 项目背景介绍.....	1
1.2 作品介绍.....	1
1.3 作品完成清单.....	3
2 作品核心功能.....	4
2.1 系统框图.....	4
2.2 步进电机梯度加减速控制.....	5
2.3 逆运动学解算.....	5
2.3.1 Delta 机械臂逆运动学解算.....	5
2.3.2 平行四边形连杆机械臂逆运动学解算.....	6
2.4 机械臂路径规划.....	6
2.5 OV5640 双目摄像头数据采集.....	7
2.6 物体形状和颜色识别.....	8
2.7 HDMI 实时显示.....	9
2.8 VL6180 距离传感器.....	9
2.9 上位机控制.....	10
3 关键模块技术分析.....	11
3.1 步进电机控制模块.....	11
3.2 Delta 机械臂控制.....	13
3.2.1 Delta 机械臂设计.....	13
3.2.2 Delta 机械臂逆运动学解算.....	14
3.2.3 Delta 机械臂逆运动学 Verilog 实现.....	18
3.3 平行四边形连杆机械臂控制.....	19
3.3.1 机械臂的设计.....	19
3.3.2 机械臂逆运动学解算.....	20
3.3.3 机械臂逆运动学 Verilog 实现.....	24
3.4 机械臂路径规划.....	25
3.4.1 Delta 机械臂线性插补算法.....	25
3.4.2 Delta 机械臂门型路径规划.....	27
3.4.3 平行四边形连杆机械臂路径规划.....	28

3.5 OV5640 摄像头数据采集.....	30
3.5.1 摄像头图像预处理.....	30
3.5.2 颜色识别与提取思路.....	30
3.5.3 Verilog 实现.....	32
3.6 物体形状识别.....	34
3.7 VL6180X 距离传感器	36
3.7.1 传感器介绍.....	36
3.7.2 软核 I2C 原理.....	36
2.7.3 软核 I2C 设计.....	38
3.7.4 传感器寄存器配置.....	39
3.8 HDMI 显示	40
3.8.1 HDMI 硬件.....	40
3.8.2 视频时序标准.....	40
3.8.3 HDMI 输出程序设计	41
3.9 上位机交互.....	43
3.9.1 上位机界面.....	43
3.9.2 功能实现.....	45
4 性能指标分析.....	46
4.1 机械臂运动控制.....	46
4.1.1 Delta 机械臂性能	46
4.1.2 平行四边形连杆机械臂性能.....	48
4.1.3 逆运动学解算.....	48
4.1.4 逆运动学解算资源优化.....	51
4.2 传感器数据处理.....	53
4.2.1 颜色与形状识别.....	53
4.2.2 物体定位精度.....	54
4.3 人机交互接口.....	55
4.3.1 上位机交互.....	55
4.3.2 语音模块实时交互.....	55
5 创新点.....	56
5.1 多机器人协同优化.....	56

5.2 多级数据放大优化 FPGA 定点运算精度	56
5.3 乘法器分时复用降低资源占用.....	56
5.4 逆运动学解算速度的创新优化.....	57
5.5 双目摄像头分区显示与实时监控.....	57
5.6 低延迟高精度距离反馈集成.....	57
6 总结.....	58
6.1 功能实现.....	58
6.2 技术创新.....	58
6.3 性能表现.....	59
6.4 应用价值.....	59
7 参考文献.....	60

1 作品介绍

1.1 项目背景介绍

随着智能制造和工业 4.0 的快速发展，自动化流水线在提升生产效率、优化资源分配以及增强生产灵活性方面的需求日益增长。分拣任务作为自动化流水线中的核心环节，直接影响生产线的整体效率和可靠性。该任务要求系统具备高实时性以应对动态工作环境、高精度以确保分类准确性，以及良好的适应性以处理多样化的物体摆放方式。然而，传统分拣系统通常依赖单一的机械结构和低速处理单元，其算法复杂性和硬件限制难以满足传送带上随机摆放物体的高效动态分拣需求。此外，传统系统在面对多类型、多姿态物体的识别与分拣时，往往存在响应延迟或误判率较高的缺陷，限制了其在现代工业生产中的广泛应用。

为解决上述问题，本项目提出了一种基于 FPGA 的双机械臂协同控制系统。该系统通过集成高分辨率 OV5640 双目摄像头以及两种机械臂（Delta 机械臂和平行四边形连杆机械臂），实现了传送带上物体的快速识别与精准分类分拣。FPGA 凭借其高并行处理能力和低延迟特性，显著提升了系统的实时性能；OV5640 摄像头提供高质量的视觉输入，结合双机械臂的协同操作，优化了分拣流程的效率和准确性。该设计为工业 4.0 背景下的自动化流水线提供了高效、可扩展的解决方案，具备较强的工程应用价值和未来升级潜力。

1.2 作品介绍

本项目以两块紫光同创 PGL22G 开发板为核心硬件平台，分别控制 Delta 机械臂、平行四边形连杆机械臂以及 OV5640 双目摄像头，构建了一个完整的物体分拣系统。

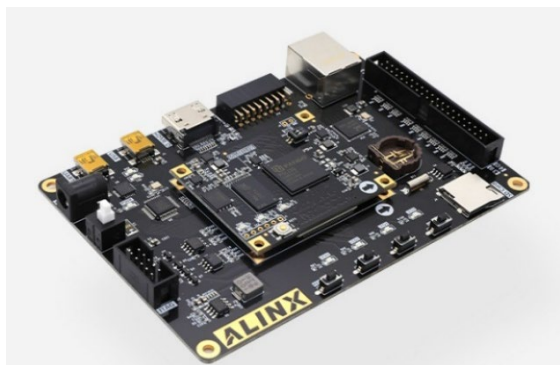


图 1.1 紫光同创 PGL22G

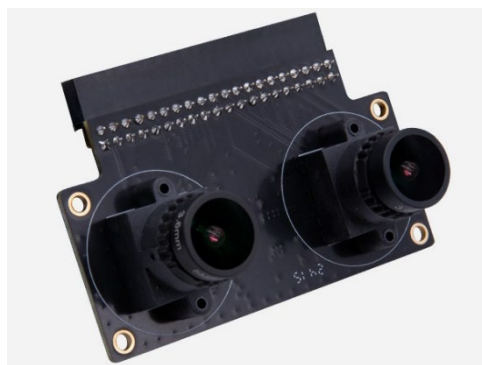


图 1.2 双目摄像头

具体实施流程如下：

首先，操作人员在初始托盘上随机摆放 12 个物体，涵盖四种颜色（红、蓝、黄、绿）和三种形状（圆形、方形、六角形），形成多样化的分拣目标。摆放完成后，托盘通过传送带运输至 OV5640 双目摄像头下方进行识别。摄像头利用立体视觉技术实时分析物体的颜色和形状，并通过 HDMI 接口输出视频流，显示识别结果，包括物体的位置、颜色和形状信息。这一阶段确保了视觉识别的高效性和直观性，为后续分拣操作提供了准确的数据基础。

识别完成后，传送带将托盘移动至 Delta 机械臂的操作区域。Delta 机械臂负责根据识别结果，依次拾取托盘上的物体，并将其按颜色分类放置至对应的目标托盘中。与此同时，平行四边形连杆机械臂协同工作，负责提供新的空托盘以支持连续分拣操作。具体而言，当 Delta 机械臂完成某一颜色物体的放置（例如将三个红色物体放入一个托盘）后，平行四边形连杆机械臂将当前托盘移走，并取回一个新的空托盘。上述过程循环进行，直至 12 个物体全部分类完成。该系统通过双机械臂的分工协作和 FPGA 的高效控制，实现了从识别到分拣的全自动化流程，显著提高了分拣效率和可靠性。

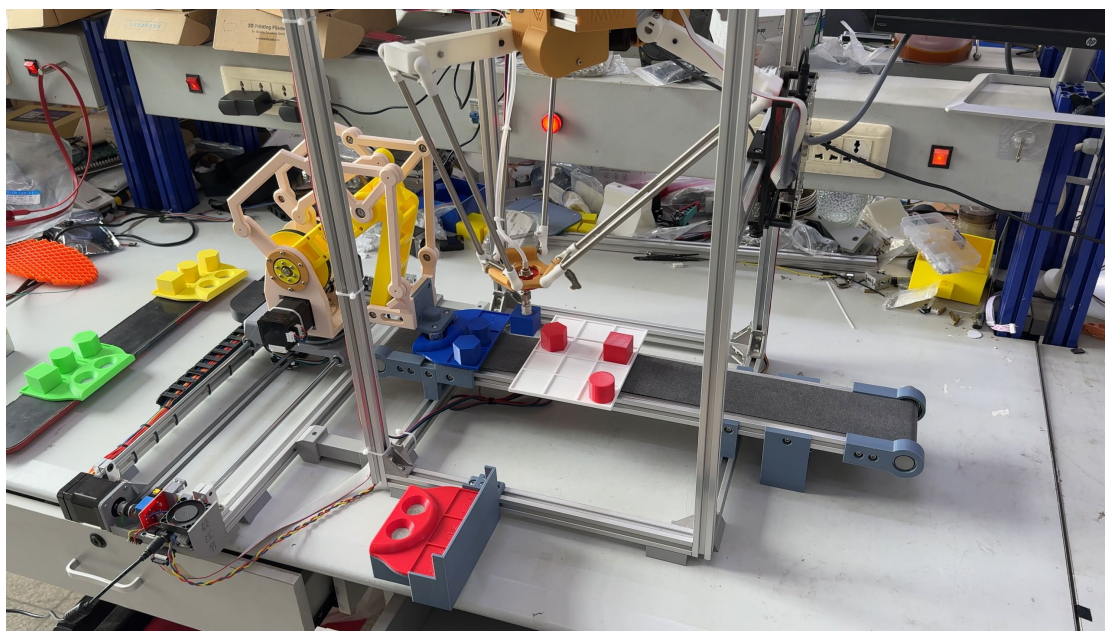


图 1.3 作品实物图

1.3 作品完成清单

根据赛题要求，本作品模拟了工业生产线上多个机器人实现低延迟、高精度协同控制的场景，成功实现了 Delta 机械臂与平行四边形连杆机械臂的智能协作分拣功能，贴近实际工业自动化应用需求。

基础任务

- **机器人运动控制**

系统实现了步进电机的梯度加减速算法控制（报告 [2.2](#) 章节）。同时基于 Verilog 实现了 Delta 机械臂与平行四边形连杆机械臂的逆运动学解算（报告 [2.3](#) 章节）。

- **传感器数据处理**

系统集成 RFID 射频标签读取功能，可准确识别托盘的唯一编号，真实模拟了工业场景中的物流运输过程。

- **人机交互接口**

上位机提供了直观的控制和监控界面，支持手动输入坐标或一键执行预设路径（报告 [2.9](#) 章节）。HDMI 显示模块处理双目摄像头数据，实时呈现物体识别结果（报告 [2.7](#) 章节）。

进阶任务

- **低延迟高精度反馈**

系统通过 I²C 接口与 VL6180X 距离传感器通信，集成于 Delta 机械臂末端，实时感知环境距离，距离数据反馈优化了吸取路径（报告 [2.8](#) 章节）。

- **感知与目标识别**

利用 FPGA 处理双目摄像头采集的 720P 数据，实现了目标物体的颜色识别与形状识别（报告 [2.5](#)、[2.6](#) 章节）。

- **多机器人协作**

Delta 机械臂与平行四边形连杆机械臂实现协同工作，Delta 机械臂负责物体拾取与分类放置；连杆机械臂通过磁吸方式搬运托盘（报告 [2.4](#) 章节）。

本作品设计充分考虑了实际工业场景的需求，结合低延迟（逆运动学解算 4.7 微秒）、高精度（定位误差 ≤ 1 毫米）和多机器人协作，成果可广泛应用于工业自动化领域，如智能分拣、物流运输和柔性制造系统，为提升生产效率和智能化水平提供了技术支持。

2 作品核心功能

2.1 系统框图

本系统采用模块化设计架构，整体功能划分为两大相互协作的板块，分别部署在两块紫光同创 PGL22G 开发板上，借助串口通信实现高效的数据交互与协同控制，确保系统运行的稳定性和实时性。

第一板块主要负责视觉识别与数据处理，集成了 OV5640 双目摄像头和 HDMI 显示器，作为系统的感知核心，负责物体的图像采集、处理与分类识别。这些识别结果不仅通过 HDMI 接口以视频流形式实时输出，供操作人员直观监控，还通过串口传输至第二板块，为机械臂分拣提供指导。

第二板块专注于机械臂的控制与路径规划，连接 Delta 机械臂和平行四边形连杆机械臂，负责执行物体的物理分拣任务。通过串口从第一板块获取物体位置和分类信息，随后路径规划模块根据这些数据为两套机械臂分别设计优化运动轨迹。路径规划过程结合逆运动学解算模块，精确计算两机械臂的关节角度，确保运动轨迹的流畅性和协同性，从而实现高效的分拣流程。

此外，系统设计中融入了人机交互功能，两块板卡均支持上位机控制，操作人员可通过 PC 软件发送指令，灵活调整机械臂的姿态、暂停分拣流程或进行手动干预。

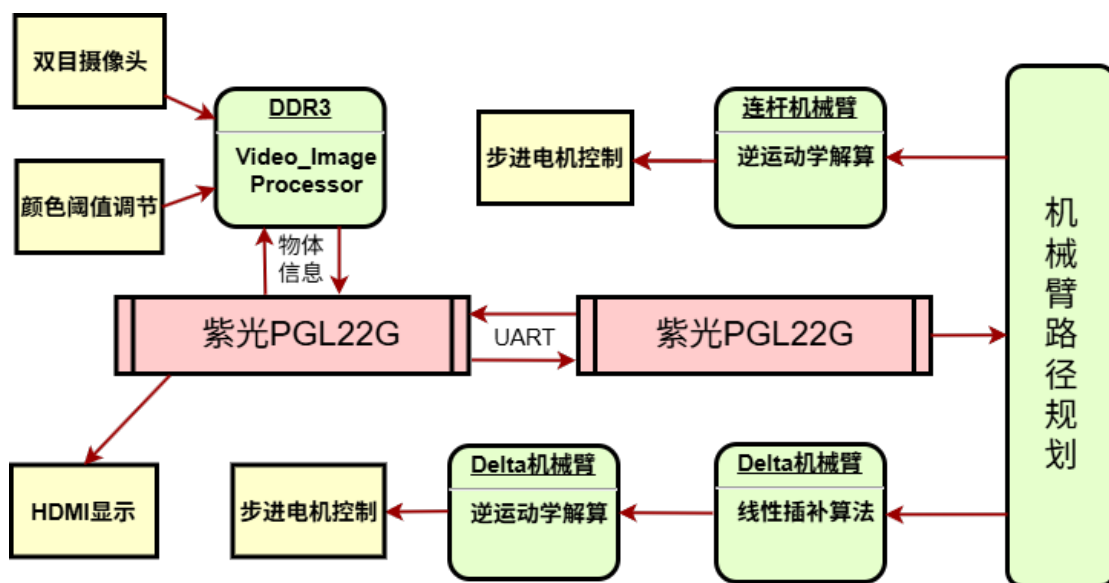


图 2.1 系统框图

2.2 步进电机梯度加减速控制

在本系统中，Delta 机械臂和平行四边形连杆机械臂均采用步进电机作为驱动单元，FPGA 通过高精度脉冲信号精确控制电机旋转角度，实现末端执行器的精准定位与运动控制。然而，初期采用固定频率脉冲信号驱动步进电机，导致运动启停阶段出现剧烈抖动，显著降低定位精度。

为解决此问题，系统在步进电机控制模块中引入梯度加减速算法，通过动态调整脉冲频率实现平滑的加速与减速过程，有效消除速度突变引发的振动和失步现象。该算法分为三个阶段：加速阶段，脉冲频率从低值逐步增至目标频率；匀速阶段，维持恒定频率确保运动连续性；减速阶段，频率渐降至零，实现平滑停止。此策略显著改善机械臂动态响应，减少因惯性或负载变化导致的机械振动。

针对 Delta 机械臂和平行四边形连杆机械臂各轴的负载特性和运动需求差异，系统为每轴定制独立加减速比例参数，通过 FPGA 灵活配置实现个性化优化。这些参数根据轴的惯量、负载重量和运动距离动态调整，确保加减速曲线匹配实际工况。

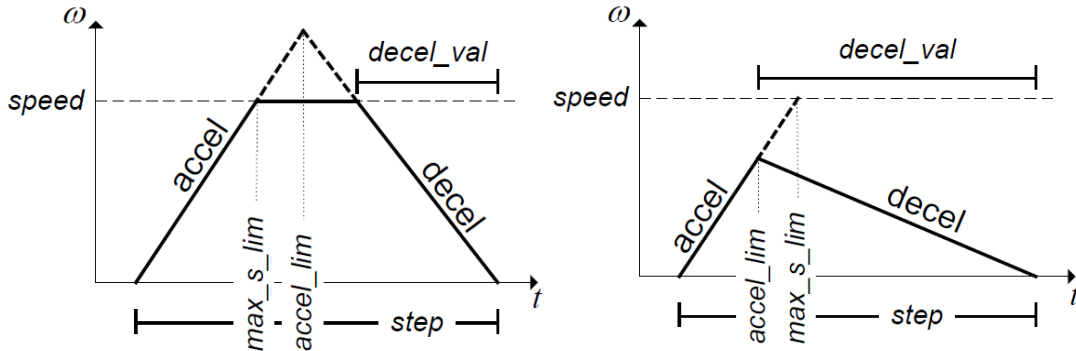


图 2.2 步进电机梯度加减速算法

2.3 逆运动学解算

2.3.1 Delta 机械臂逆运动学解算

Delta 机械臂是系统核心执行部件，负责物体精准抓取与分类放置，其运动控制依赖高效的逆运动学解算模块。逆运动学解算的目标是将末端执行器三维空间目标位置 (X, Y, Z) 映射为三个驱动臂的关节角度，涉及复杂的几何计算和反三角函数运算。为此，系统采用 CORDIC 算法，通过串行计算实现高效反三角函数求解，仅需 17 个时钟周期即可输出高精度结果，兼顾计算稳定性和时间效率。

逆运动学解算需大量乘法运算，对 FPGA 算术资源要求较高。系统集成紫光同创乘法器 IP 核以加速矩阵运算和坐标变换，但 PGL22G 芯片仅配备 30 个 DSP 乘法器，难以支持全并行运算。为优化资源利用，设计采用分时复用技术，将有限 DSP 资源分配给不同计算阶段，通过时序调度和流水线设计实现高效计算。尽管计算速度略有下降，但在 PGL22G 芯片上成功实现了逆运动学解算，确保了性能与资源的平衡。

2.3.2 平行四边形连杆机械臂逆运动学解算

平行四边形连杆机械臂是系统辅助组件，负责托盘抓取与更换，满足工业流水线高效搬运需求。相比传统三轴机械臂，其新增 Y 轴平移轴，显著扩展末端执行器运动范围，适应多场景托盘操作。该机械臂采用串联-并联混合结构，逆运动学解算需同时计算三个旋转关节角度和一个平移轴位移，计算复杂度较高。

为应对挑战，系统沿用紫光同创乘法器 IP 核的分时复用策略，并集成专用除法器模块优化算术运算。除法器采用硬件加速算法，仅需 16 个时钟周期完成 32 位除法。逆运动学解算通过几何模型解析末端位置与关节角度关系，结合分时复用和流水线技术同步计算四个电机目标角度。

2.4 机械臂路径规划

路径规划是机械臂控制系统的核心，直接影响运动轨迹平滑性、分拣精度和系统稳定性。Delta 机械臂和平行四边形连杆机械臂分别针对其运动特性和任务需求，采用路径规划策略，实现高效协同操作，满足工业自动化流水线分拣要求。

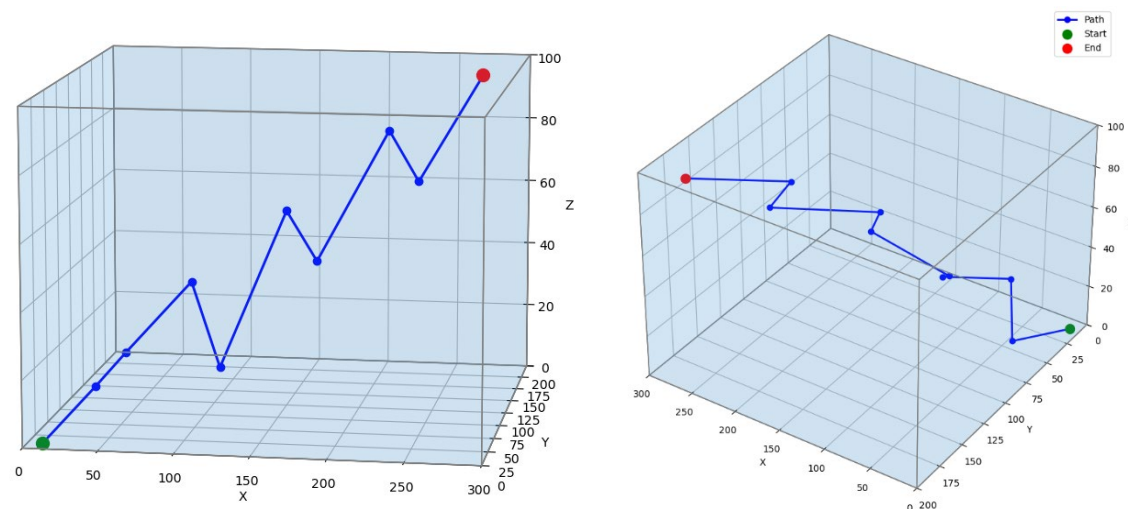


图 2.3 路径规划示意图

(1) Delta 机械臂直线线性插补

Delta 机械臂负责从传送带抓取物体并按颜色分类放置。为确保末端执行器三维空间运动平滑性，需解决各轴位移差值不一致引发的轨迹抖动问题。初期测试显示，位移较小轴提前到达目标点，导致末端执行器偏离直线路径，降低分拣稳定性和增加机械磨损风险。为此，设计引入直线线性插补算法，将各轴运动路径分为 64 段（利用 FPGA 移位操作特性），采用整数运算替代浮点运算，降低资源消耗。各轴步进步长根据最大轴差值同步调整，确保末端执行器沿直线平滑移动。优化后，轨迹平滑性显著提升，抖动现象几乎不可见，系统可靠性大幅增强。

(2) 平行四边形连杆机械臂路径规划

平行四边形连杆机械臂负责托盘抓取与更换，需在铝材框架限制的狭窄空间内实现精准操作，面临碰撞风险和复杂运动约束。设计引入专用路径规划模块，通过优化大臂和小臂姿态，动态调整运动角度，避开框架并精确引导末端执行器到达目标位置。在托盘切换中，模块根据实时位置和姿态动态调整，兼顾安全性和定位精度，将定位误差控制在 $\pm 1\text{mm}$ 以内。该方案有效降低碰撞风险，提升机械臂在复杂环境下的适应性和稳定性，为协同分拣任务提供可靠保障。

2.5 OV5640 双目摄像头数据采集

OV5640 摄像头是系统视觉识别的核心组件，负责从传送带托盘内采集 12 个随机摆放物体的图像，为物体识别与定位提供高质量数据支持。该摄像头采用 $1.4\mu\text{m} \times 1.4\mu\text{m}$ 像素尺寸，集成 OmniBSI 背照式感光技术，具有高灵敏度、低串扰和低噪声特性，能在复杂光照条件下输出清晰、稳定图像，适合工业视觉应用。其捕获的原始图像为后续颜色与形状识别提供可靠输入，确保分拣任务的准确性和实时性。

系统基于紫光同创 PGL22G 开发板，采用双目 OV5640 摄像头配置以实现立体视觉，获取物体深度信息，提升定位精度。通过 I²C 接口初始化配置 OV5640，设置输出格式为 720P@30fps 的 RGB565 图像数据。RGB565 格式以 16 位存储像素，兼顾图像质量与数据传输效率，适配 FPGA 高速处理能力。FPGA 内部实现高效数据采集模块，结合双目摄像头例程，通过并行处理实时接收和缓存图像数据流。为应对工业环境光照变化，代码中包含了上位机通过串口调节颜色阈值，提升识别模块鲁棒性。采集数据经 FPGA 高速总线实时传输至图像处理模块，为物体识别、定位和高精度分拣提供稳定输入。以下是摄像头拍摄的原始图像。

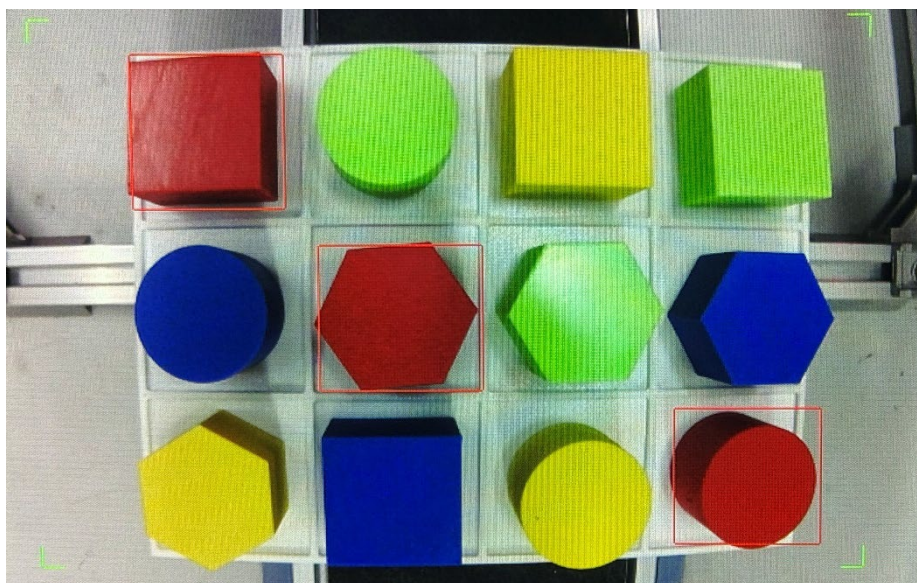


图 2.4 摄像头画面

2.6 物体形状和颜色识别

物体形状与颜色识别模块是本系统视觉处理的核心组成部分，负责从 OV5640 摄像头采集的 720P RGB565 图像中提取 12 个随机摆放物体的颜色和形状特征，为 Delta 机械臂和平行四边形连杆机械臂提供精准定位数据，直接影响分拣任务的准确性和效率。为实现高效处理，该模块设计分为预处理和识别两个子模块，分别负责图像优化和特征提取。

预处理模块旨在提升图像质量，增强识别效果。首先将 RGB565 图像转换为 HSV 颜色空间，依据预设的红、蓝、黄、绿阈值提取目标区域。随后进行二值化处理，并根据噪声水平动态应用 2 至 5 次腐蚀操作，有效去除背景干扰，增强物体边缘轮廓，为后续识别提供清晰输入。

识别模块基于优化的车牌识别算法，结合面积计算和色块分离策略，实现高效物体初步分离，降低 FPGA 资源消耗。通过边界检测提取物体外接矩形框及几何中心，获取形状参数，准确区分圆形、正方形和六边形。

图 2-4 展示了物体形状与颜色识别的实际效果。该图包含四种颜色（红、蓝、黄、绿）和三种形状（圆形、正方形、六边形）的物体，反映了托盘内 12 个物体的随机摆放场景。图像的四个角以绿色线框标注，框定了识别范围，表明算法的工作区域。在识别范围内，红色方框精确框出识别到的红色物体，体现了模块在颜色和形状识别上的准确性，同时绿色线框与红色方框的结合直观展示了识别算法的定位能力，为系统的视觉处理性能提供了直观验证。

2.7 HDMI 实时显示

HDMI 实时显示模块在系统视觉识别与机械臂协同控制中至关重要，负责将 OV5640 双目摄像头采集的 720P 图像数据实时传输至外部 HDMI 显示设备，为操作人员提供直观监控界面。

FPGA 内部集成 HDMI 控制器，通过 TMDS 编码将 RGB565 格式图像数据转换为 HDMI 1.4 标准高速差分信号，输出至显示设备。为实现双目摄像头数据同步显示，设计调用两个 video_rect_read_data 模块，将屏幕划分为左右区域：左侧显示左目摄像头经二值化处理的图像，突出物体轮廓和颜色特征，便于评估预处理效果；右侧显示右目摄像头数据，结合识别算法结果，以画框标注物体颜色、形状及空间坐标。

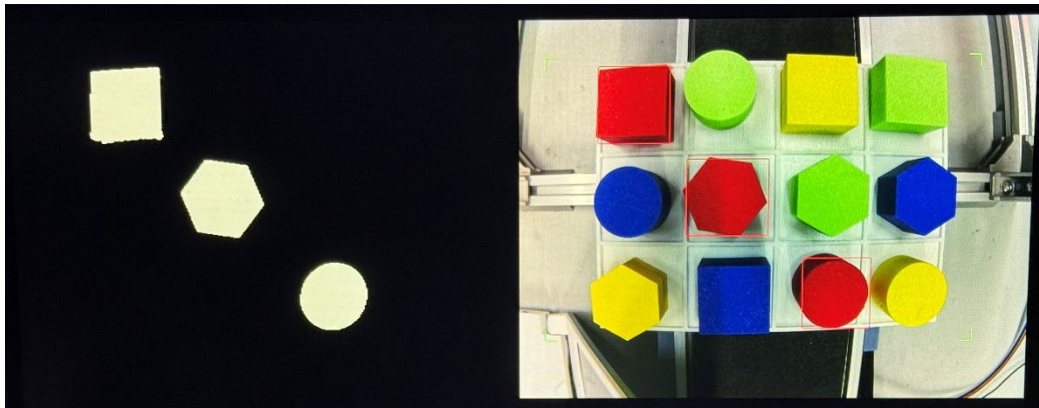


图 2.5 HDMI 显示画面

2.8 VL6180 距离传感器

VL6180X 测距传感器是系统高精度定位的核心组件，专为 Delta 机械臂末端吸盘与托盘上 12 个随机摆放物体之间的距离测量设计，确保吸取动作的精准性和稳定性。该传感器采用飞行时间 (ToF) 技术，测量范围 0~100 mm，标称精度 ± 1 mm，支持高频采样，适配动态近距离定位场景，为 Delta 机械臂吸取操作提供可靠数据支持。

为实现与 VL6180X 的通信，系统在紫光同创 PGL22G FPGA 平台上开发了 PC 软核模块，通过 Verilog 代码实现传感器配置 (如单次测距模式) 和数据采集。单次测距耗时约 1 ms，PC 传输延迟优化后总响应时间低于传统串口方案。FPGA 内集成的 PC 控制模块通过优化时钟同步和数据缓冲机制，确保数据采集高效、无丢失。

为增强环境适应性，系统通过 VL6180X 实现机械臂环境感知功能，结合多点采样与中值滤波算法（连续采集 5 次数据并取中值），有效抑制振动和光照干扰引起的噪声，测量误差优化至 $\pm 1\text{ mm}$ 。高精度距离反馈使 Delta 机械臂吸盘根据物体实际高度精确调整下降距离，吸取失败率从约 5%降至低于 1%，显著提升分拣任务的效率和成功率。

2.9 上位机控制

为提升机械臂系统的操作性与精度，我们设计并开发了一款基于 Qt 框架的上位机软件，用于简化 Delta 机械臂控制流程，提供高效、直观的人机交互体验。

软件界面采用 QML 编写，具备声明式布局和动态响应能力，实现了实时数据显示、按钮操作与系统状态反馈等功能。QML 与 C++ 的无缝集成，使软件兼具良好的用户体验与强大的后台处理能力。

上位机通过标准 UART 协议与紫光同创 PGL22G FPGA 板卡通信，支持通过按钮或坐标输入发送控制指令，如姿态调整、任务切换等。指令经 FPGA 快速解析后，机械臂可在毫秒级内响应操作请求。

此外，软件支持实时显示机械臂的状态信息，包括关节角度、末端坐标、分拣进度及识别结果等，帮助操作人员全面掌握系统运行状况。借助 QML 的动态界面刷新能力与 C++ 的高效底层通信机制，该软件在复杂工业分拣场景中运行稳定，有效提升了系统操作的精度与便捷性。



图 2.6 上位机界面

3 关键模块技术分析

3.1 步进电机控制模块

步进电机控制模块驱动双机械臂的步进电机，FPGA 生成脉冲信号实现精准角度控制。为解决固定频率控制导致的启动与停止抖动，我们开发了梯度加减速算法，动态调整脉冲频率，确保平滑运动。

对于步进电机，其步进角度 α ，位置 θ ，速度 ω 有以下公式：

$$\alpha = \frac{2\pi}{spr}(\text{rad}) \quad \theta = n\alpha(\text{rad}) \quad \omega = \frac{\alpha}{\delta t}(\text{rad/sec})$$

其中 spr 为每圈步数， n 是步数， δt 为脉冲间隔时间。

为实现平稳的启动和停止，需控制加速度和减速度，我们可以采用恒定加速度/减速度产生线性速度曲线，即可以得到下图：加速度、位置与时间的关系。

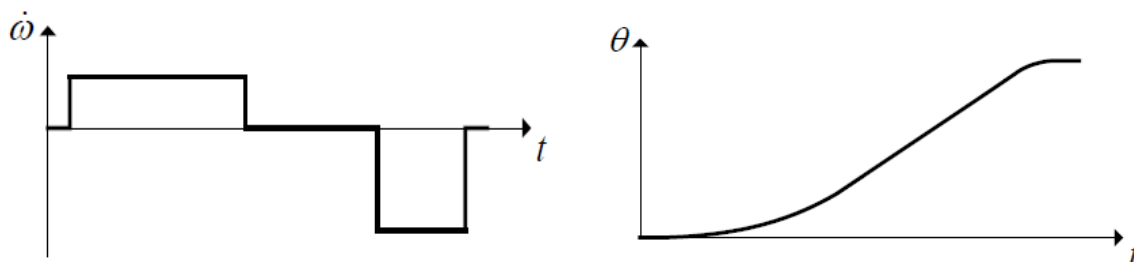


图 3.1.1 加速度和位置与时间的关系

其中脉冲间隔时间 δt 直接影响电机速度，需根据目标速度计算，为了得期望的速度曲线，可以看到下图的对比。

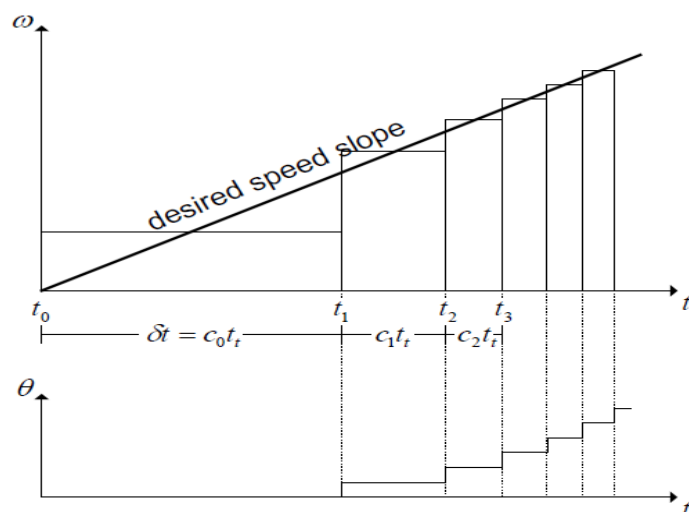


图 3.1.2 理想速度曲线与实际步进脉冲生成的速度曲线的对比

由图中，我们看到初始计数器延迟 c_0 和后续延迟 c_n 可以通过以下公式计算：

$$c_0 = \frac{1}{t_t} \sqrt{\frac{2\alpha}{\omega}} \quad c_n = c_0(\sqrt{n+1} - \sqrt{n})$$

但是这些计算涉及大量的浮点运算，不仅会浪费 FPGA 大量的资源，也会损失步进电机的精度。

为降低计算复杂度，我们提出了使用泰勒级数近似计算延迟：

$$c_n = c_{n-1} - \frac{2c_{n-1}}{4n+1}$$

当 $n=1$ 时，这种近似方法引入的误差仅为 0.44，与直接计算 c_0 的误差为 0.876 相比，精度较高且计算更简单。

以上是数学理论推导，接下来我们带入到实际 Verilog 代码。

在步进电机控制模块（stepper_motor_ctrl）的设计中，系统通过 50MHz 系统时钟驱动步进电机，以实现基于输入目标角度的精确旋转方向、速度和位置控制，并确保运动过程中的平滑加速、匀速和减速。为了提升控制精度，输入角度被放大 256 倍，通过放大倍数保留更多浮点数据以减少量化误差。

模块内定义了步进电机的关键参数，以适配不同机械臂的应用场景：**Delta 机械臂**：脉冲周期范围为 5000 至 10000，对应 50MHz 时钟下约 5kHz 至 10kHz 的步进频率。**平行四边形机械臂**：脉冲周期范围为 6000 至 1000，对应 50MHz 时钟下约 6kHz 至 1kHz 的步进频率。针对不同轴的电机，设置了独立的加减速参数，以优化机械臂末端的运动稳定性。

为简化数学推导中涉及的复杂计算（如平方根运算），模块引入了加速步长机制，即每隔一个脉冲后调整脉冲周期。运动过程被划分为三个阶段：**加速阶段**：脉冲周期从最大 MAX_PULSE_PERIOD 逐步减小至最小值 MIN_PULSE_PERIOD，实现速度递增；**匀速阶段**：脉冲周期保持在 MIN_PULSE_PERIOD，以维持最高速度运行；**减速阶段**：脉冲周期从 MIN_PULSE_PERIOD 逐步增大至 MAX_PULSE_PERIOD，实现速度平滑下降。

其中加速和减速步数根据总步数动态分配：若总步数小于两倍加减速步数，则加速和减速各占一半，无匀速阶段，否则，加速和减速步数固定，剩余步数为匀速阶段。

在 ModelSim 仿真中得到如下波形图：

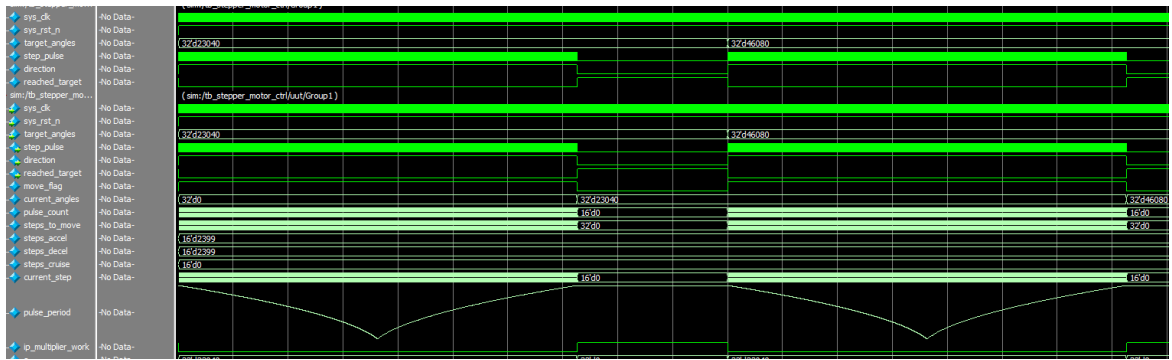


图 3.1.3 步进电机梯度加减速仿真波形

在图 3.1.3 中可以看到，控制步进电机的脉冲周期逐渐减小（速度变快）之后再逐渐增大（速度减小），由于仿真总步数小于两倍加减速步数，故波形中没有体现匀速运动部分。

3.2 Delta 机械臂控制

3.2.1 Delta 机械臂设计

Delta 机械臂是一种高性能的并联机器人，采用轻量化结构和并联设计，减少了运动部件的惯性，其中电机和驱动单元通常安装在固定基座上，移动部分仅包括末端执行器和连接杆，降低了整体惯性，提高了能效和响应速度。其设计特点使其在工业自动化领域中表现出色，特别适用于需要快速、精确操作的场景。

出于经济性和稳定性，我们使用 SolidWorks 自主设计并打印 Delta 机械臂。

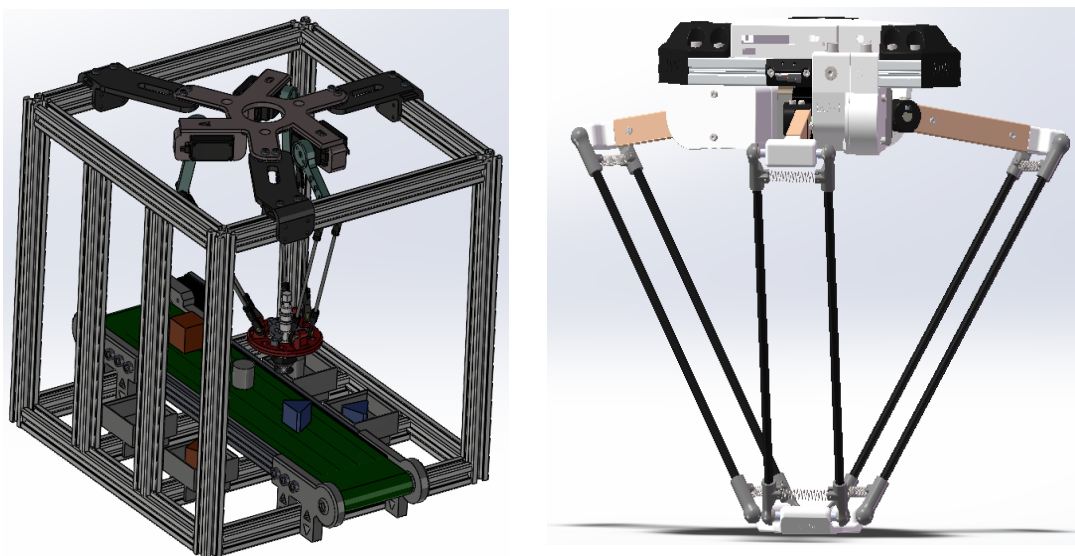


图 3.2.1 Delta 机械臂模型

在机械臂设计中，我们定义了 Delta 机械臂相关的参数：主动臂长 130mm，从动臂长 316mm，静平台半径 74.69mm，动平台半径 34.64mm。之后便可计算机械臂的工作空间，其代表了机器人的活动范围，是衡量机器人工作能力的一个重要指标。机器人的工作空间定义为：在结构限制下末端执行器能够到达的所有位置的集合。

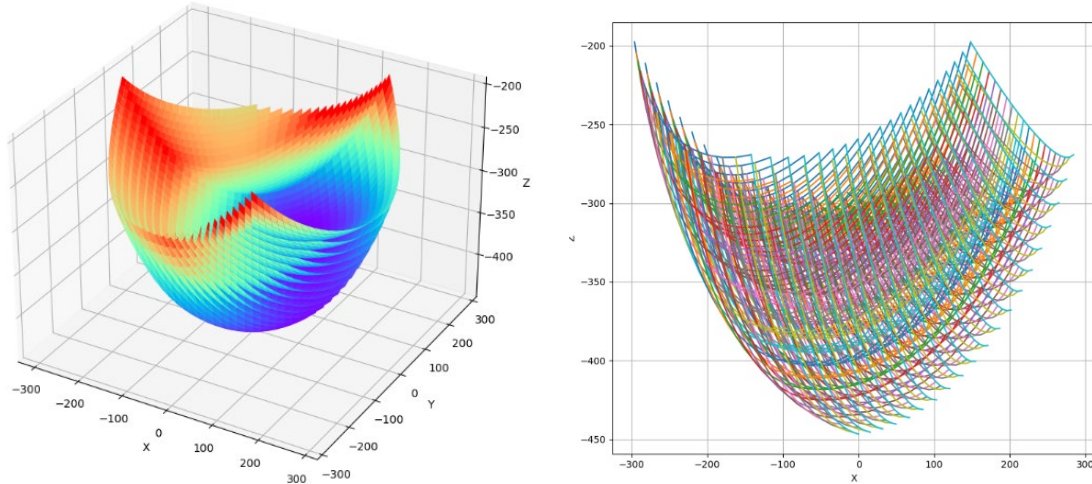


图 3.2.2 Delta 机械臂工作空间

使用 python 程序可以计算得到我们的机械臂工作空间为 X 轴和 Y 轴为-170mm 到 170mm，Z 轴为-200mm 到-400mm。由此，我们搭建了一个 400*400*600mm 的铝材框架，确保机械臂能正常工作。

3.2.2 Delta 机械臂逆运动学解算

逆运动学解算作为 Delta 机械臂控制的核心技术，旨在根据已知的末端执行器在三维空间中的位置和姿态，精确计算机械臂各关节变量的过程，也被称为机器人运动学的求解。

(1) 结构解析

Delta 机械臂采用独特的并联机构设计，其结构由多个关键部件协同构成，具有高刚性、高速性和高精度的特点。具体构件包括：**固定基座**：我们采用铝材框架，将机械臂安装在铝材框架顶部。**活动平台（末端执行器）**：位于机械臂底部，放置吸盘以及 VL6180X 距离传感器。**三组对称的运动链**：每组运动链由一个驱动臂（主动臂）和两个平行连杆（从动臂）组成，通过球铰链或万向节连接基座和活动平台。**驱动系统**：使用三个步进电机，安装在固定基座上，通过运动链控制活动平台的运动。

将主动臂和从动臂抽象成一根线，静平台与动平台抽象成三角形，那么可以得到 Delta 机械臂的数学模型：

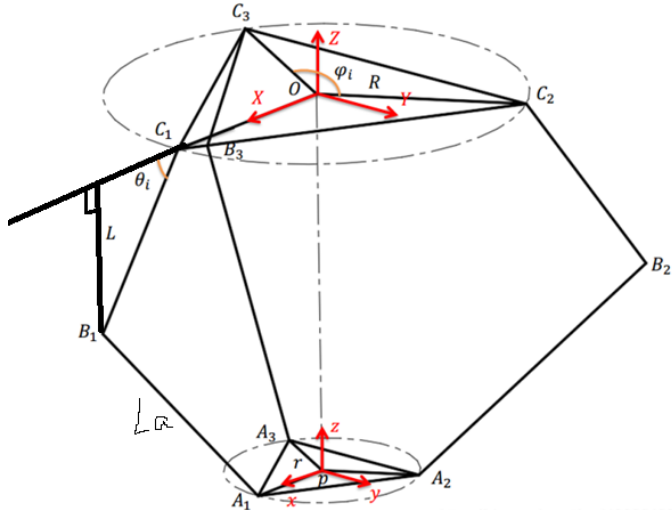


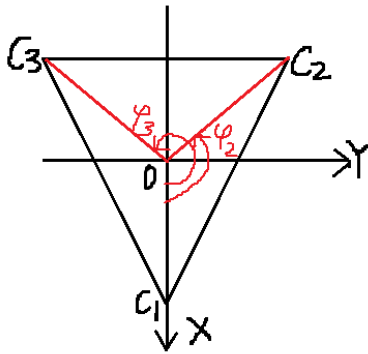
图 3.2.3 Delta 结构数学简化模型

图中对应的参数含义如下表：

表 3.2.1 机械臂模型参数

结构参数	数学模型参数	含义
$ OC_i $	R	静坐标系原点 O 到静平台各顶点的长度
$ pA_i $	r	动坐标系原点 p 到动平台各顶点的长度
$\vec{op}$	$[x \ y \ z]^T$	动坐标系的原点相对于静坐标系的位置矢量
$ C_iB_i $	L	主动臂的长度
$ B_iA_i $	L_a	从动臂的长度
θ_i	θ_i	主动臂与静平台夹角

(2) 运动学方程分析



静坐标系与静平台的空间几何关系如图所示。

则 C_i 在静坐标系 O -XYZ 中的位置矢量为：

$$OC_i = \begin{bmatrix} R \cos \varphi_i \\ R \sin \varphi_i \\ 0 \end{bmatrix}$$

$$\varphi_i = \frac{i-1}{3} \pi, (i=1,2,3)$$

图 3.2.4 静平台坐标系

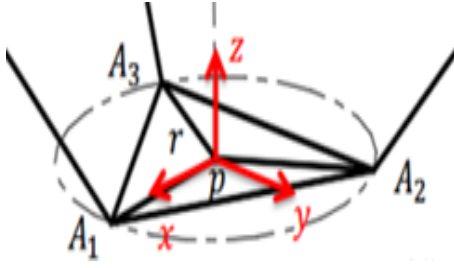


图 3.2.5 动平台坐标系

同样也可以得到点 A_i 在动坐标系

p - $X'Y'Z'$ 中的位置矢量:

$$pA_i = \begin{bmatrix} r \cos \varphi_i \\ r \sin \varphi_i \\ 0 \end{bmatrix}$$

$$\varphi_i = \frac{i-1}{3} \pi, (i=1,2,3)$$

根据几何关系, 可以得到点 B_i 在静坐标系 O - XYZ 中的位置矢量为:

$$OB_i = \begin{bmatrix} (R + L \cos \theta_i) \cos \varphi_i \\ (R + L \cos \theta_i) \sin \varphi_i \\ -L \sin \theta_i \end{bmatrix} \quad \varphi_i = \frac{i-1}{3} \pi, (i=1,2,3)$$

Op 为动坐标系的原点 p 相对于静坐标系的矢量

$$Op = [x \ y \ z]^T$$

则点 A_i 在静坐标系 O - XYZ 中的位置矢量为:

$$OA_i = Op + pA_i = \begin{bmatrix} r \cos \varphi_i + x \\ r \sin \varphi_i + y \\ z \end{bmatrix} \quad \varphi_i = \frac{i-1}{3} \pi, (i=1,2,3)$$

因此可以得到 $A_i B_i$ 的矢量表达式为:

$$A_i B_i = OB_i - OA_i = \begin{bmatrix} (R + L \cos \theta_i - r) \cos \varphi_i - x \\ (R + L \cos \theta_i - r) \sin \varphi_i - y \\ -L \sin \theta_i - z \end{bmatrix}$$

最后, 根据 $A_i B_i = L a$, 可推导出运动学方程:

$$\begin{cases} f(\theta) = (R + L \cos \theta_i - r) \\ [f(\theta) \cos \varphi_i - x]^2 + [f(\theta) \sin \varphi_i - y]^2 = L a^2 \end{cases}$$

(3) 逆运动学解算

得到运动学方程之后, 便可进行反解。

已知机械臂末端的空间坐标 (X, Y, Z) , 需要求解每一个电机的角度。

即 $(\theta_1, \theta_2, \theta_3)$, 其中 $\varphi_i = \frac{2i-2}{3} \pi$, $(i=1,2,3)$ 。

$$\begin{aligned} & [(R + L\cos\theta_i - r)\cos\varphi_i - x]^2 + \\ & [(R + L\cos\theta_i - r)\sin\varphi_i - y]^2 + [-L\sin\theta_i - z]^2 = La^2 \end{aligned} \quad (1)$$

1、将 $\varphi_1 = 0$, $\varphi_2 = \frac{2}{3}\pi$, $\varphi_3 = \frac{4}{3}\pi$ 分别代入①式得：

$$[(R + L\cos\theta_1 - r) - x]^2 + y^2 + (L\sin\theta_1 + z)^2 = La^2 \quad (2)$$

$$\begin{aligned} & [-\frac{1}{2}(R + L\cos\theta_2 - r) - x]^2 + \\ & [\frac{\sqrt{3}}{2}(R + L\cos\theta_2 - r) - y]^2 + (L\sin\theta_2 + z)^2 = La^2 \end{aligned} \quad (3)$$

$$\begin{aligned} & [-\frac{1}{2}(R + L\cos\theta_3 - r) - x]^2 + \\ & [-\frac{\sqrt{3}}{2}(R + L\cos\theta_3 - r) - y]^2 + (L\sin\theta_3 + z)^2 = La^2 \end{aligned} \quad (4)$$

2、将 x , y , z 代入②③④式中，并化简，得：

$$A1 = 2L(r - R + x)$$

$$B1 = -2zL$$

$$C1 = (r - R + x)^2 + y^2 + z^2 + L^2 - La^2$$

$$A2 = 2L(r - R - 0.5x + 0.8660y)$$

$$B2 = -2zL$$

$$C2 = (r - R - 0.5x + 0.8660y)^2 + (0.8660x + 0.5y)^2 + z^2 + L^2 - La^2$$

$$A3 = 2 * (r - R - 0.5 * x - 0.8660 * y) * m$$

$$B3 = -2 * z * m$$

$$C3 = (r - R - 0.5x - 0.8660y)^2 + (0.8660x - 0.5y)^2 + z^2 + L^2 - La^2$$

3、以上是对空间坐标 (X , Y , Z) 的预处理，得到结果之后，有一个通式如下：

$$A\sin\theta + B\cos\theta = C$$

对于这个通式的通解为：

$$\tan\theta = \frac{-AB - C\sqrt{A^2 + B^2 - C^2}}{A^2 - C^2}$$

最后使用 $\arctan\theta$ 即可求出每一个电机的角度值。

3.2.3 Delta 机械臂逆运动学 Verilog 实现

在 Verilog 代码的实现中，Delta 机械臂的逆运动学解算被划分为三个功能模块，以实现空间坐标到电机角度的精确转换。

逆运动学解算的实现首先从 `pretreat_data` 模块开始，该模块负责对输入的空间坐标 (X, Y, Z) 进行预处理，遵循逆运动学推导的第二步（如文档 3.2.2 所述）。为提升定点运算的精度并避免有效数据的丢失，输入的 X、Y、Z 坐标被放大 16 倍，而输出的中间参数（称为 ABC 参数）进一步放大 65536 倍（即 2^{16} ）。若不进行这种倍数放大，定点运算的精度限制将导致计算过程中显著的有效数据丢失，从而影响解算结果的准确性。基于推导的数学公式，`pretreat_data` 模块将原始坐标代入抽象表达式进行计算。例如，原始公式为：

$$C1 = (x + r - R)^2 + y^2 + z^2 + m^2 - n^2$$

在放大 65536 倍后，公式转换为：

$$C1 = (16x - 10253)^2 + (16y)^2 + (16z)^2 - 5436604416$$

这种放大数据的处理方式不仅有效避免了精度损失，还简化了复杂表达式的计算过程，使其更适合 Verilog 的定点运算和硬件实现。放大后的数据通过移位操作实现，进一步提升了计算效率，同时增强了模块的模块化程度，为后续计算单元提供了高质量的输入。

`calculate_angle` 模块实现了逆运动学解算的第三步推导，专注于计算每个驱动电机的目标旋转角度。该模块基于推导得到的通式，集成了多种计算单元以完成复杂的数学运算：首先，乘法器单元用于处理坐标的平方和加减运算，确保快速完成公式中的多项式计算；其次，开平方器单元专门负责计算涉及距离的平方根操作，例如上述公式中的中间变量计算；最后，CORDIC 算法模块通过坐标旋转数字计算机算法高效实现角度的迭代计算，输出精确的电机角度。CORDIC 算法采用串行计算方式，显著优化了硬件资源的利用率，同时保证了计算的实时性。经测试，该模块能够在 14 个时钟周期内完成一次角度计算。

`motion_solver` 模块作为逆运动学解算的顶层模块，采用三段式有限状态机（FSM）设计，负责协调 `pretreat_data` 和 `calculate_angle` 模块的执行流程，确保整个解算过程的有序性和高效性。三段式 FSM 依次调用预处理和角度计算模块，并将最终输出的角度数据传输至步进电机控制模块，从而完成完整的逆运动学解算。

3.3 平行四边形连杆机械臂控制

3.3.1 机械臂的设计

平行四边形连杆机械臂是一种基于平行四边形机构的机械结构，与 Delta 机械臂相同，广泛应用于工业机器人和自动化设备中，其设计特点使其在运动稳定性和负载能力方面具有独特优势。

同样，此机械臂我们也使用 SolidWorks 自主设计并打印平行四边形连杆机械臂控制。

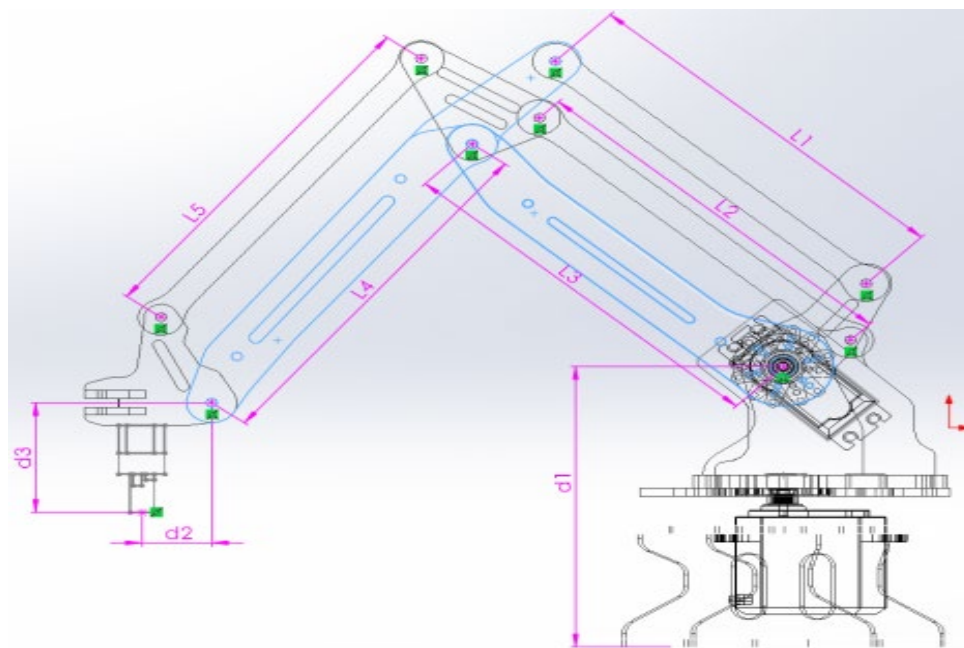


图 3.3.1 机械臂尺寸草图

在机械臂的设计上，本系统采用了平行四边形连杆机构的四轴结构，这一设计通过独特的几何特性显著简化了逆运动学解算过程，同时确保机械臂末端执行器在运动过程中能够维持固定的姿态。

然而，传统的平行四边形连杆机械臂在执行物品托盘的拿取和放置任务时，其运动范围受限于机械结构的固有约束，难以满足分拣场景中对较大工作区域的需求。为此，我们在传统三轴结构的基础上，特意为机械臂的 Y 轴增加了一根平移轴，通过引入额外的步进电机实现了四轴控制。Y 轴运动范围为-300 毫米至 700 毫米，提供了充足的纵向移动能力以应对托盘在传送带上的不同位置；X 轴运动范围为-300 毫米至 300 毫米，支持机械臂在水平方向上的灵活移动；Z 轴运动范围为-100 毫米至 270 毫米，确保末端执行器在垂直方向上能够完成从托盘拾取到放置的全程操作。

3.3.2 机械臂逆运动学解算

对机械臂的位置逆解算法的实现，可以使用 D-H 法、几何法进行分析求解。其中几何解析法，根据连杆型码垛机械臂现有的几何关系进行分析，并建立方程求解。在图 3.3.2、图 3.3.3 所示的坐标系链中，以第 0 坐标系的视角为基准，以平行于地面作 XOY ，垂直于地面并穿过 O 点作 Z 轴。

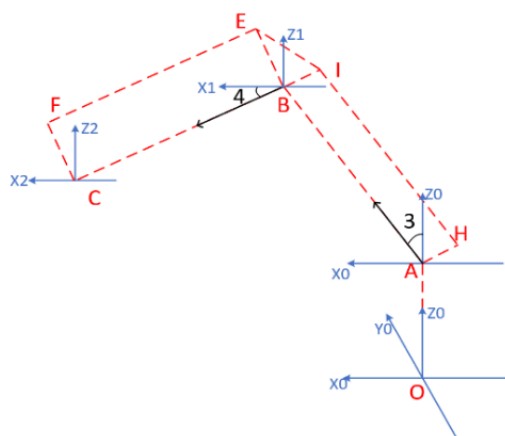


图 3.3.2 右视图坐标系链

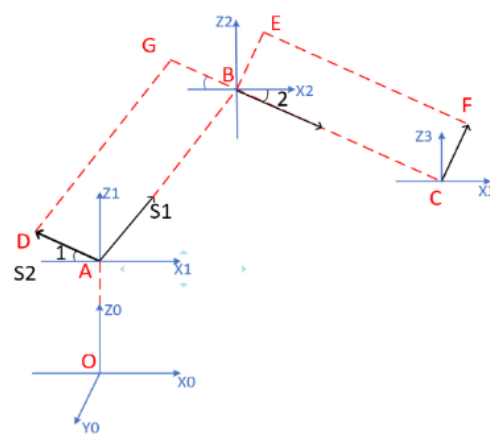


图 3.3.3 左视图坐标系链

此时假设底座步进电机 S_0 初始位置为指向 $+X$ 轴；大臂步进电机 S_1 的初始位置为指向 $+Z$ 轴；小臂步进电机 S_2 初始位置为指向机械臂末端的反向。

则有如下图 3.3.4、图 3.3.5 所示。

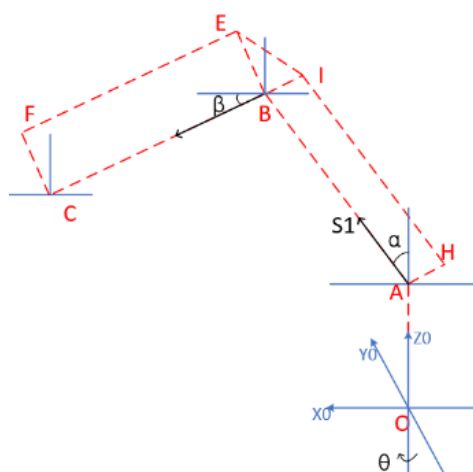


图 3.3.4 右视图几何分析模型

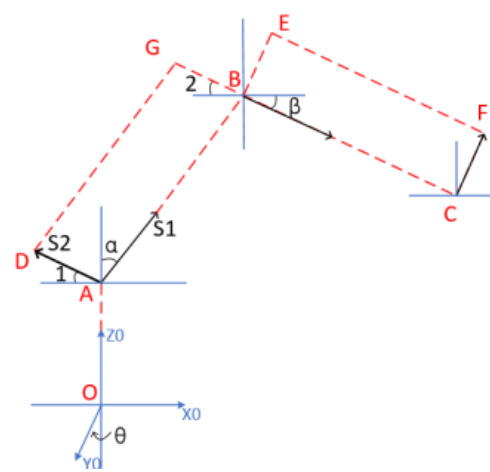


图 3.3.5 左视图几何分析模型

$\angle\theta$ 为 S_0 偏转角， $\angle\alpha$ 为 S_1 偏转角， $\angle 1$ 为 S_2 偏转角

由机械臂构型中，以左视图 3.3.5 中， $ABDG$ 和 $BCEF$ 构成两个平行四边形，以其几何特性，可以得到： $\angle 2 = \angle\beta$ ， $\angle 1 = \angle 2$ ；

在图 3.3.4 中，两个平行四边形 ABIH、BCFE 随着大臂的角度变化而变化，而机械臂的末端执行器由于平衡三角(EBI)的长度、内角皆恒定而保持水平。

综合下来，采取几何解析法，只需求取的角为： $\angle \theta$ 、 $\angle \alpha$ 、 $\angle \beta$ 。

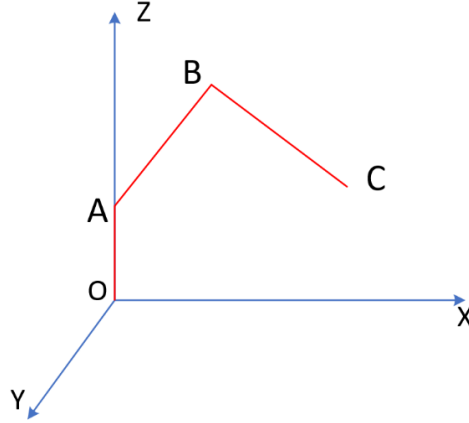


图 3.3.6 机械臂的模型简化

图 3.3.6 是几何解析法的思想对机械臂的模型简化。

这里，我们设定：已知 C 点坐标 (x, y, z) ，AB 为大臂长度 L_1 ，BC 为小臂长度 L_2 ，到这里，我们需要求解三个量：

1. 转台转动的角度（记为步进电机 S0 角度）
2. 大臂转动的角度（记为步进电机 S1 角度）
3. 小臂转动的角度（记为步进电机 S2 角度）

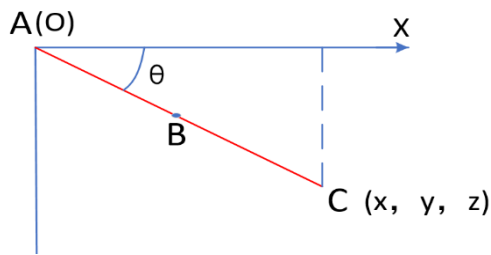


图 3.3.7 俯视图

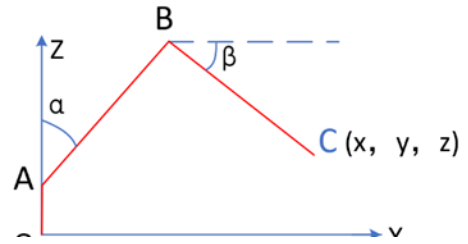


图 3.3.8 平面投影

(a) 以机械臂俯视图看，如图 3.3.7，AB、BC 分别为大臂和小臂的平面投影， θ 则为转台转动的角度，此时有关系式：

$$\tan \theta = \frac{y}{x}$$

由此可得转台角度 θ ，

$$\theta = \arctan\left(\frac{y}{x}\right)$$

(b) 以机械臂侧视图看，如图 3.3.8，将大臂 AB、小臂 BC 投影于机械臂所在的平面，可得出需求解大臂角 α 、小臂角 β 。

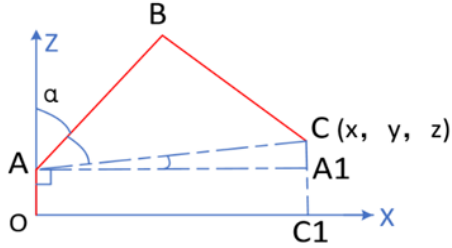


图 3.3.9 $CC1 > OA$

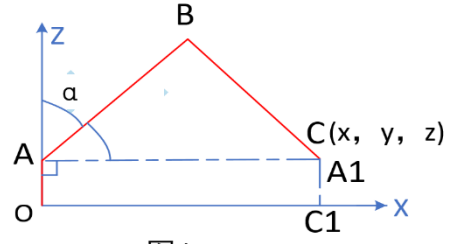


图 3.3.10 $CC1 = OA$

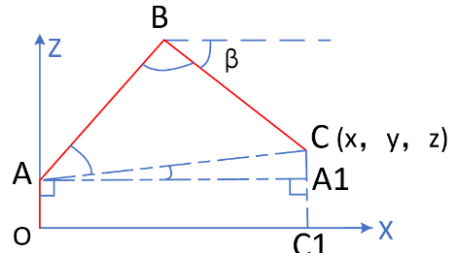


图 3.3.11 $CC1 > OA$

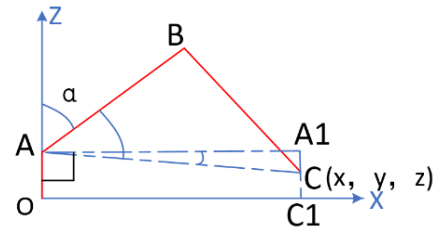


图 3.3.12 $CC1 < OA$

首先求解大臂角 α ：

如图 3.3.9，做一条垂直于 Z 轴的线 $CC1$ 与 X 轴相交于 $C1$ 点，做一条平行于 X 轴，与 A 点重合的线 $AA1$ ，相交 $CC1$ 于 $A1$ 点。

$$\tan(\angle CAA1) = \frac{CA1}{AA1} = \frac{CA1}{OC1};$$

由 C 点坐标可得，

$$OC1 = \sqrt{x^2 + y^2}$$

$$\angle CAA1 = \arctan\left(\frac{CA1}{OC1}\right) = \arctan\left(\frac{|Z - OA|}{\sqrt{x^2 + y^2}}\right)$$

大臂 AB、小臂 BC 已知，

余弦定理求解 $\angle AOB$ ：

$$\cos(\angle BAC) = \frac{AB^2 + AC^2 - BC^2}{2 \times AB \times AC}$$

$$\angle BAC = \arccos\left(\frac{AB^2 + AC^2 - BC^2}{2 \times AB \times AC}\right)$$

其中 $OA = L1, AB = L2, OA = d1$

此时有 3 种情况，

① 当 $CC_1 > OA$ 时，有图 3.3.9 所示，

$$\alpha = \frac{\pi}{2} - \angle BAC - \angle CAA_1$$

$$\alpha = \frac{\pi}{2} - \arccos\left(\frac{AB^2 + AC^2 - BC^2}{2 \times AB \times AC}\right) - \arctan\left(\frac{|Z - OA|}{\sqrt{x^2 + y^2}}\right)$$

② 当 $CC_1 = OA$ 时，则有图 3.3.10 所示，

此时， AA_1 与 CA 重合，

$$\angle CAA_1 = 0$$

$$\alpha = \frac{\pi}{2} - \angle BAC$$

$$\alpha = \frac{\pi}{2} - \arccos\left(\frac{AB^2 + AC^2 - BC^2}{2 \times AB \times AC}\right)$$

③ 当 $CC_1 < OA$ 时，有图 3.3.12 所示，

$$\alpha = \frac{\pi}{2} - (\angle BAC - \angle CAA_1)$$

$$\alpha = \frac{\pi}{2} - \arccos\left(\frac{AB^2 + AC^2 - BC^2}{2 \times AB \times AC}\right) - \arctan\left(\frac{|Z - OA|}{\sqrt{x^2 + y^2}}\right)$$

其次求解小臂角 β ：

依如图 3.3.11，做一条平行于 Z 轴的线 CC_1 与 X 轴相交于 C_1 点，做一条平行于 X 轴，与 A 点重合的线 AA_1 ，相交 CC_1 于 A_1 点，同时过 B 点做平行线可得到角 β 。

此时可得，

$$\beta = \pi - \angle CAA_1 - \angle BAC - \angle ABC$$

由余弦定理得，

$$\angle ABC = \arccos\left(\frac{AB^2 + BC^2 - AC^2}{2 \times AB \times BC}\right)$$

其中，在求解大臂角 α 时，已得 $\angle CAA_1, \angle BAC$ ，故

$$\beta = \pi - \arctan\left(\frac{|Z - OA|}{\sqrt{x^2 + y^2}}\right) - \arccos\left(\frac{AB^2 + AC^2 - BC^2}{2 \times AB \times AC}\right) - \arccos\left(\frac{AB^2 + AC^2 - BC^2}{2 \times AB \times BC}\right)$$

3.3.3 机械臂逆运动学 Verilog 实现

在平行四边形连杆机械臂的 Verilog 实现中，其逆运动学解算相比 Delta 机械臂更为复杂，涉及的运算量显著增加，计算过程也更为繁琐，主要由于其四轴结构（包括大臂、小臂、旋转轴及 Y 轴平移轴）需要同时满足多关节耦合关系和运动范围约束。为提升模块化设计水平、优化硬件资源利用率并保证计算的实时性，整个逆运动学解算过程被划分为三个子模块：

`solver_big_and_small_arm`、`solver_rotation` 和 `arm_motion_solver`。

`solver_big_and_small_arm` 模块负责对输入的空间坐标 (X, Y) 进行计算，其中输入的 X 是真实的 X 坐标与 Y 坐标的平方和开根号的结果，而输入的 Y 则是真实的 Z 坐标值。这一模块的核心任务是计算机械臂大臂角和小臂角的数值。大臂角定义为大臂与 X 轴负半轴之间的夹角，顺时针方向为正；小臂角则为大臂与小臂之间的夹角，逆时针方向为正。基于文档 3.2.2 的数学推导，设计将复杂的逆运动学公式抽象化，使其更适合 Verilog 的定点运算和硬件实现。与 Delta 机械臂类似，该模块集成了多种计算单元，包括乘法器用于处理坐标的平方和运算、开平方器用于计算距离的平方根，以及 CORDIC 算法模块用于高效迭代计算角度。此外，为应对平行四边形连杆机械臂更复杂的几何关系，模块新增了一个 32 位宽的有符号数除法器，该除法器仅需 16 个时钟周期便可完成一次除法运算。

`solver_rotation` 模块负责处理输入的空间坐标 (X, Y)，其中 X 为真实的 X 坐标，Y 为真实的 Y 坐标，其主要任务是利用 CORDIC 算法模块高效计算旋转轴电机的目标旋转角度。由于平行四边形连杆机械臂紧贴 Delta 机械臂的铝材框架运行，存在潜在的碰撞风险，为此模块在角度计算后增加了额外的旋转方向约束。根据计算得到的旋转角度，系统明确规定了顺时针或逆时针的旋转方向，确保机械臂在运动过程中能够避开铝材框架的物理障碍。这一方向约束有效提升了系统在狭窄空间内的安全性，同时保持了运动轨迹的精确性。

`arm_motion_solver` 作为逆运动学解算的顶层模块，采用了三段式有限状态机 (FSM) 设计，通过状态机协调 `solver_big_and_small_arm` 和 `solver_rotation` 模块的执行流程。该 FSM 设计分为初始化状态、计算状态和输出状态三个阶段，分别负责模块初始化、角度计算以及数据传输至步进电机控制模块。FSM 通过时序管理实现了大臂、小臂和旋转轴三个轴电机角度的同步解算，最大限度减少了计算延迟，并确保了机械臂各关节的协调运动。

3.4 机械臂路径规划

3.4.1 Delta 机械臂线性插补算法

路径规划旨在为 Delta 机械臂末端执行器生成从起始点到目标点的运动轨迹，满足速度、加速度、避障和机械约束要求。Delta 机械臂的运动学特性要求路径规划综合考虑三维平移自由度和几何约束。其工作空间呈圆锥形，受驱动臂、连杆长度及关节角度范围限制，增加了规划复杂性。此外，机械臂的高速运动特性要求路径具备平滑的加速度和减速度曲线，以避免振动和失步。

路径规划以逆运动学解算（参考文档 3.2）为基础，将目标位置转化为电机角度序列，并引入时间维度生成连续运动轨迹。常见方法包括直线插补和样条曲线插补。样条曲线插补通过构造三次样条曲线，提供连续的速度和加速度曲线，适合复杂轨迹规划，但其高计算复杂度对 FPGA 的 APM 资源消耗较大，尤其在逆运动学解算已占用大量资源的情况下。直线插补则通过在笛卡尔空间线性划分路径点，确保末端执行器沿直线移动，适用于简单点到点运动，避免复杂浮点运算，显著降低资源占用。

为适配紫光同创 PGL22G FPGA 的有限资源，系统选用直线插补算法。算法将路径分为等距段（通常为 64 段，优化移位操作），通过整数运算替代浮点运算，确保计算高效且精度满足要求。规划过程同步调整各轴步进长度，匹配最大轴差值，保障轨迹平滑性。测试表明，该方案有效抑制运动抖动，末端执行器定位误差控制在 $\pm 1\text{mm}$ 以内，满足高速分拣任务的平滑性和稳定性要求。

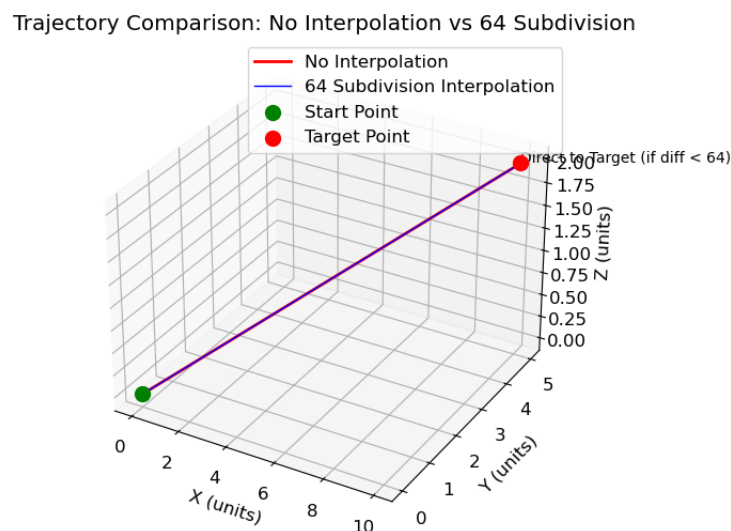


图 3.4.1 未加算法的机械臂末端运动轨迹

在实际实现中，Delta 机械臂的路径规划结合硬件控制和软件算法，旨在生成平滑、精准的末端执行器运动轨迹，满足速度、加速度和机械约束要求。路径规划模块与步进电机控制模块协作，通过逆运动学解算和动态脉冲周期调整，实现平滑点到点运动。

在基于 Verilog 的控制系统中，路径规划模块将目标路径分解为离散采样点，通过逆运动学解算模块计算每个采样点对应的电机角度。以 Delta 机械臂为例，其步进电机脉冲周期范围为 5000 至 10000（对应 50MHz 时钟下 5kHz 至 10kHz 频率）。为确保平滑运动，模块预计算加速度和减速度阶段的步数：加速阶段脉冲周期从最大值渐减至最小值，匀速阶段维持最小值，减速阶段渐增至最大值。此策略避免速度突变，显著提升末端执行器稳定性。

```
if(delta_x_flag) begin
    current_x <= start_x + ((delta_x * current_step * INV_INTERP_STEPS) >> 16);
end
if(delta_y_flag) begin
    current_y <= start_y + ((delta_y * current_step * INV_INTERP_STEPS) >> 16);
end
if(delta_z_flag) begin
    current_z <= start_z + ((delta_z * current_step * INV_INTERP_STEPS) >> 16);
end
```

图 3.4.2 线性插补 Verilog 代码

在 Verilog 实现中，路径规划采用 64 细分直线插补策略，将目标位置 (X,Y,Z) 与当前位置的差值分为 64 等份，生成中间坐标点。通过定点数乘法和移位操作实现线性插值，避免复杂浮点运算，适配紫光同创 PGL22G FPGA 的资源限制。中间坐标点逐一输入逆运动学解算模块，计算电机角度，并由步进电机控制模块驱动运动。

为避免细分带来的精度丢失，设计对差值小于 64 的坐标点跳过插补，直接驱动至目标点。同时，最后一个执行点强制设为目标坐标，消除累计误差。相比未插补的直接运动，64 细分策略确保末端执行器沿直线轨迹移动，减少轨迹偏差和机械振动。

相比未使用插补的直接运动方式，线性直线插补确保了机械臂末端沿点到点的直线轨迹运动，减少了轨迹偏差和机械振动。这种方法在笛卡尔空间中通过线性划分路径点，确保了轨迹的几何一致性，同时结合脉冲周期的动态调整，实现了速度和加速度的平滑过渡。结果表明，采用 64 细分直线插补后，机械臂末端的运动轨迹更加平顺，特别是在拾取与放置等点到点任务中，末端执行器的定位精度和稳定性均得到了显著改善。

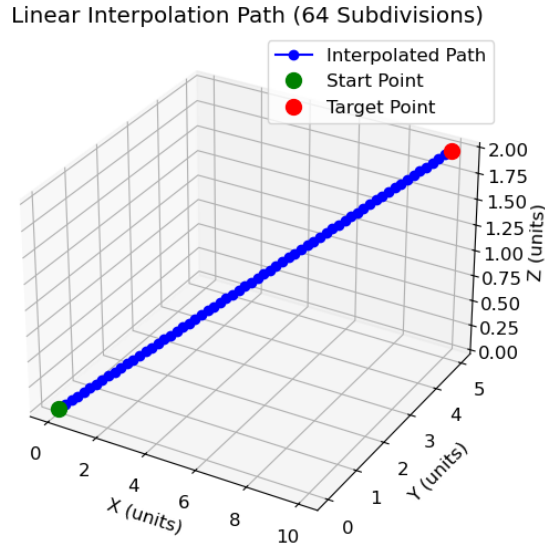


图 3.4.3 线性插补运动轨迹

3.4.2 Delta 机械臂门型路径规划

为实现高效、安全的拾放任务，Delta 机械臂采用门型轨迹从拾取点运动至放置点。门型轨迹包含三个阶段：

- (1) 竖直上升段，沿 Z 轴从拾取点上升至安全高度，避开障碍物；
- (2) 水平移动段，沿 X-Y 平面移动至放置区域上方；
- (3) 竖直下降段，沿 Z 轴下降至放置点，完成物体放置。

此轨迹通过中继点设计，有效避免与环境（如物体或框架）的碰撞。然而，门型轨迹各阶段衔接点存在方向突变，导致末端执行器在转角处经历加速度突变，引发机械振动，影响稳定性、定位精度并可能加速机械疲劳。为缓解此问题，系统结合步进电机控制中的梯度加减速算法，通过动态调整脉冲周期实现速度平滑过渡，降低振动。同时，优化运动速度以平衡效率与稳定性。

门型轨迹控制通过 Verilog 模块化设计实现，集成于路径规划模块中，作为顶层模块协调轨迹生成与运动执行。模块接收目标空间坐标 (X, Y, Z)，生成门型轨迹的坐标点序列，包括上升段安全高度点、水平段过渡点和下降段目标点。轨迹生成采用有限状态机 (FSM) 控制，工作流程如下：

1. 生成中继点坐标，传递至逆运动学解算模块，计算对应电机角度。
2. 步进电机控制模块驱动末端执行器运动至中继点，完成后返回到位信号。
3. FSM 切换至下一状态，生成后续中继点坐标，重复上述过程，直至到达目标点，完成吸取或放置。

为提升平滑性，轨迹各阶段结合前述 64 细分直线插补策略，将路径点细分为 64 等份，通过定点数运算和移位操作生成中间坐标，确保轨迹几何一致性。测试表明，门型轨迹结合梯度加减速和 64 细分插补后，末端执行器定位误差控制在 $\pm 1\text{mm}$ 以内，振动显著减小，拾放任务成功率提升至 90%以上。

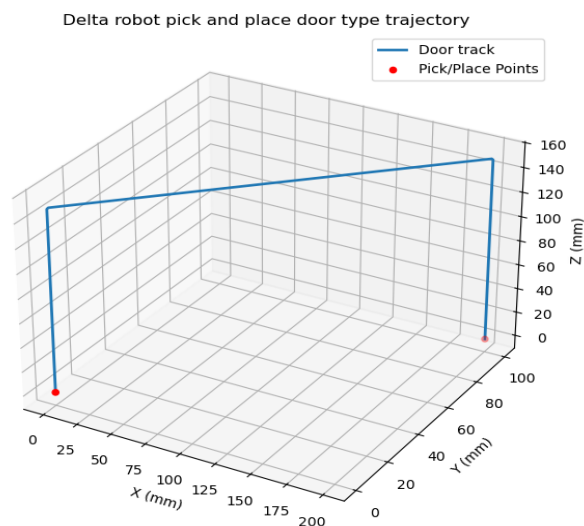


图 3.4.4 门型轨迹运动示意图

3.4.3 平行四边形连杆机械臂路径规划

尽管平行四边形连杆机械臂的逆运动学解算因其复杂的几何约束和多自由度特性而相对困难，其路径规划在本工程中的实现却较为简洁，设计重点在于满足特定任务需求。本项目中，平行四边形连杆机械臂的主要功能是执行托盘的拿取和放置操作，模拟工业流水线上的分拣和分装任务。其目标是实现不同类型物体的分拣与装载，服务于自动化生产线的效率提升和劳动力替代。与 Delta 机械臂相比，该机械臂的应用场景对末端执行器的精度和稳定性要求较低，主要关注任务的完成可靠性和操作的连续性，因此未对路径规划施加过高的实时性和复杂性要求。

考虑到紫光同创 PGL22G FPGA 的 APM 资源限制和机械臂较低的运动速度要求，本设计未在平行四边形连杆机械臂的控制中引入线性插补等高精度算法。相反，路径规划采用基于状态机的控制方法，通过定义一系列离散状态逐步管理机械臂的姿态和运动过程。具体而言，设计了 26 个状态，每个状态对应机械臂从起始位置到目标位置的特定姿态或动作阶段。这种状态机驱动的路径规划方法简化了计算复杂度，避免了对浮点运算和细分插补的依赖，从而降低了硬件资源消耗，适配了当前控制系统的性能限制。

通过状态机的设计，平行四边形连杆机械臂能够实现姿态的连续控制，从而确保运动过程的精确性和可靠性。在项目中，机械臂的运动范围受限于整体铝材框架的物理约束。铝材框架位于机械臂的操作区域内，任何运动失控都可能导致碰撞事故。为此，精确的姿态控制成为避免碰撞的关键需求，考虑到机械臂需在有限的空间范围内完成托盘的拿取和放置任务，尤其是在铝材框架的狭窄区域中操作，传统路径规划方法需进一步优化以适应这一复杂环境。

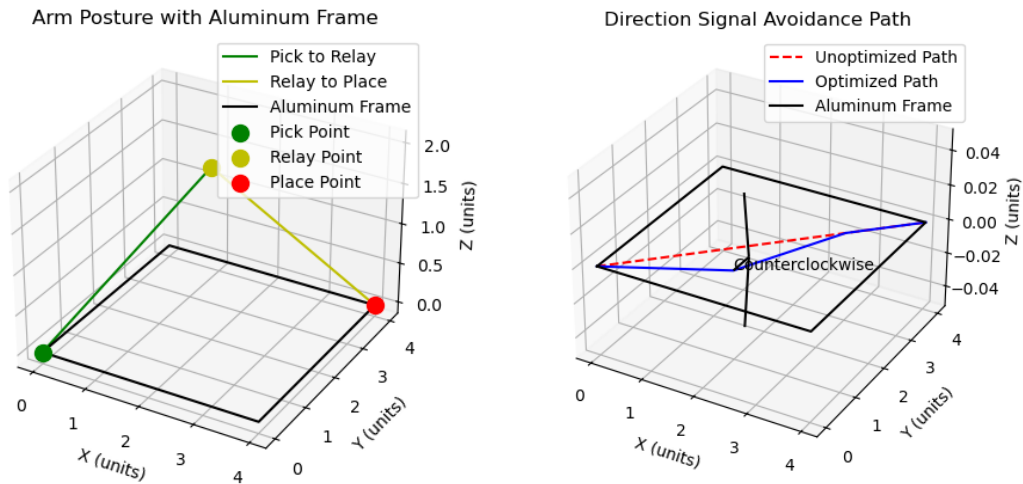


图 3.4.5 机械臂路径规划

为解决上述问题，本设计在机械臂控制系统中引入了旋转轴电机的方向信号机制。即旋转轴电机的控制不仅包括目标角度的计算，还明确规定了到达目标角度的旋转方向，即逆时针旋转或顺时针旋转。通过在 Verilog 代码中为旋转轴添加方向信号，系统能够在运动规划中动态选择最优路径。例如，当机械臂检测到铝材框架可能干扰其运动轨迹时，方向信号可触发逆时针或顺时针旋转，以绕过障碍物。这种方向选择的灵活性显著增强了机械臂在狭窄空间中的避障能力，使其能够安全、有效地完成托盘的拿取和放置任务。

状态机通过 26 个状态协调姿态调整和路径移动，每个状态对应特定的姿态调整或位置移动，例如下降至托盘位置、水平移动通过框架区域以及上升至放置点，结合旋转轴的方向信号，状态机在关键转折点（如通过框架时）根据实时姿态信息选择合适的旋转方向，确保机械臂末端始终保持安全距离，避免与铝材框架发生接触。此外，方向信号的引入还优化了运动效率，减少了不必要的回转动作，从而缩短了整体操作时间。

3.5 OV5640 摄像头数据采集

3.5.1 摄像头图像预处理

在本项目中，为方便后续对图像处理，使用了前置的图像预处理，在预处理阶段，色彩转换的思想，将不同颜色的色块直接提取出来，从而实现将 12 个薄片分成 4x3 组需要识别的色块，在一帧截取的图片中，只有三个色块，经过二值化处理后，可以得到只含相同颜色的背景与色块图，如下图所示：

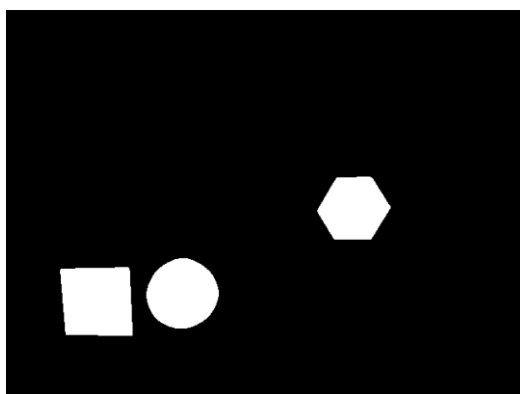


图 3.5.1 HSV 提取蓝色 H 分量

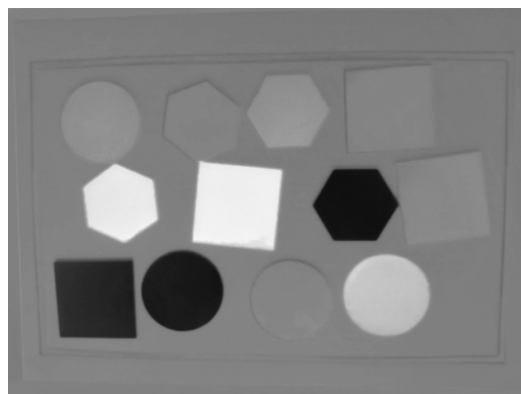


图 3.5.2 蓝色分量二值化

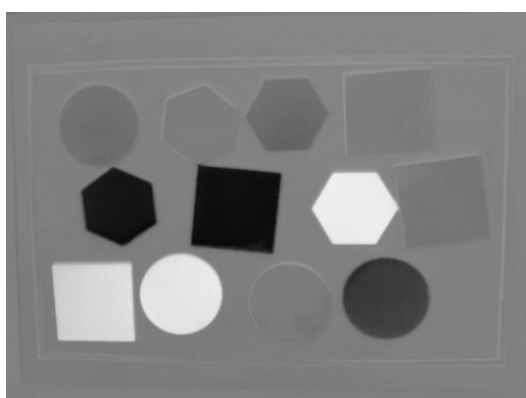


图 3.5.3 HSV 提取黄色 H 分量

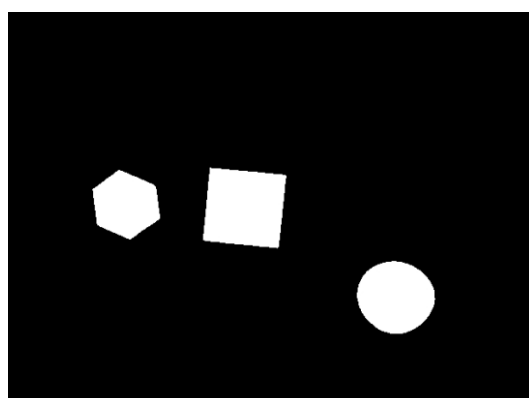


图 3.5.4 黄色分量二值化

3.5.2 颜色识别与提取思路

在数字图像处理领域，RGB 色彩空间是广泛使用的颜色表示方法，其通过 Red（红色）、Green（绿色）和 Blue（蓝色）三个分量来定义颜色，每个分量的取值范围在理论上为[0%, 100%]，反映了颜色的相对强度。然而，在计算机系统中，大多数基于 RGB 色彩空间的程序采用 1 字节（8 位）来存储每个分量，此时每个分量的取值范围转换为我们常见的[0, 255]，本文后续部分也将沿用这一取值范围。

这种表示方式直观且便于硬件实现，但其固有缺陷在于无法有效区分颜色属性中的色度、亮度和饱和度等信息。在本项目中，视觉识别模块需要精确提取物体的颜色特征（红、蓝、黄、绿），而 RGB 色彩空间由于其分量耦合性，在处理不同色度时存在困难，因此引入了 HSV 色彩空间模型以克服这一局限。

HSV (Hue, Saturation, Value) 色彩空间由 A. R. Smith 于 1978 年提出，基于颜色的直观特性构建，也被称为六角锥体模型 (Hexcone Model)。在 HSV 模型中，颜色由三个独立参数共同定义：色度 (Hue, H) 表示颜色的种类，通常以 0° 至 360° 的圆形角度表示；饱和度 (Saturation, S) 表示颜色的纯度，范围为 $[0, 1]$ ，数值越大颜色越鲜艳；明度 (Value, V) 表示颜色的亮度，同样范围为 $[0, 1]$ ，数值越大颜色越亮。相较于 RGB 模型，HSV 空间更符合人类视觉感知，能够直观地分离色度、饱和度和亮度信息，为颜色识别提供了便捷的处理基础。图 3.5.5 展示了 HSV 色彩空间模型的几何表示：

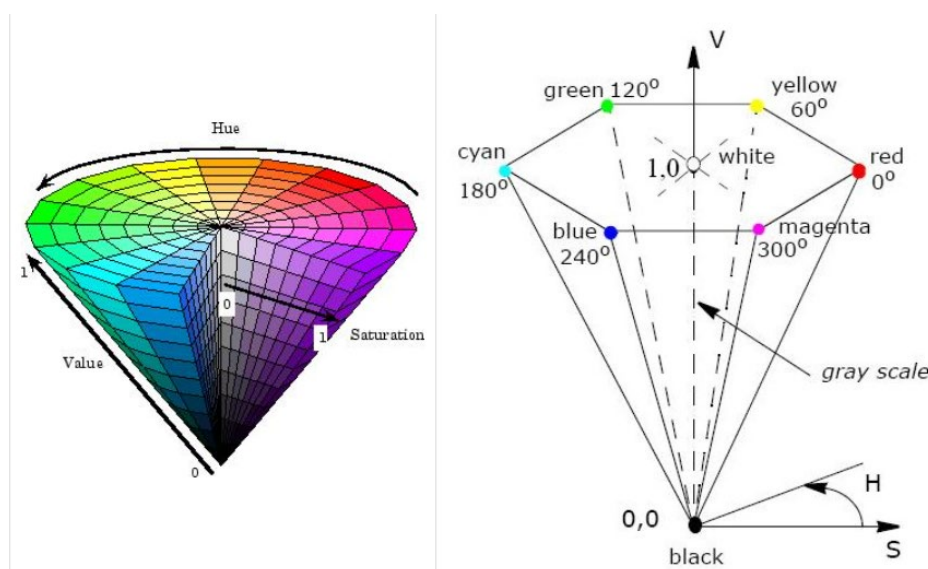


图 3.5.5 HSV 色彩空间模型

在本项目中，HSV 模型的引入主要利用色度 (H) 分量来区分不同的颜色。通过预设特定的色度阈值范围，例如红色的 H 值范围约为 0° 至 30° 和 330° 至 360° 、蓝色的 H 值范围约为 180° 至 240° 、黄色的 H 值范围约为 30° 至 90° 、绿色的 H 值范围约为 90° 至 150° ，系统能够有效分离目标物体的颜色特征。为实现颜色转换和提取，设计首先将 OV5640 摄像头采集的 RGB565 图像数据转换为 HSV 空间，随后通过修改 H 分量实现颜色变换。例如，将红色的 H 值 (0° - 30° 或 330° - 360°) 和黄色的 H 值 (30° - 90°) 调整至蓝色的 H 值范围 (180° - 240°)，再将修改后的 HSV 数据转换回 RGB 格式。

如下是通过颜色 H 值改变达到改变物体颜色的目的，如下：



图 3.5.6 20 元本色、通过改变 H 值后的 20 元、50 元对比图

3.5.3 Verilog 实现

颜色识别模块是系统视觉处理的核心，负责从 RGB565 图像中提取目标颜色色块，简化后续物体形状（圆形、正方形、六边形）和位置识别。相较于 RGB 色彩空间，HSV 模型通过色度（H）、饱和度（S）和明度（V）分离颜色特征，适合颜色提取。HSV 将颜色映射到 0° - 360° 色度区间，如红色（ 0° - 30° ， 330° - 360° ）、黄色（ 30° - 90° ）、蓝色（ 180° - 240° ）、绿色（ 90° - 180° ），便于精准区分。

该模块通过将 RGB565 图像转换为 HSV 空间，并调整色度（H）分量，将目标颜色（红、黄、绿）统一映射到蓝色区间（ 180° - 240° ），从而简化颜色提取。例如，黄色（ 30° - 90° ）通过增加约 150° 色度，红色（ 0° - 30° ， 330° - 360° ）增加约 180° 色度，均映射至蓝色区间。转换后的图像再转回 RGB 空间，通过提取蓝色分量实现颜色分离。这一方法显著降低了颜色识别的复杂度，为形状识别提供清晰输入。

为提升模块的灵活性和复用性，设计了三个参数接口：SELECT_BIT、ADD_VALUE 和 THRESHOLD。SELECT_BIT 选择灰度通道；ADD_VALUE 控制色度增量（如 150 用于黄色转蓝色，180 用于红色转蓝色）；THRESHOLD 设置二值化阈值（如 100，适配工业环境噪声）。例如，配置 SELECT_BIT=2、ADD_VALUE=150、THRESHOLD=100 可高效提取黄色物体。这种参数化设计使模块通过简单配置即可适应多种颜色识别需求。

模块核心逻辑依赖于色度调整和阈值切换机制，确保在复杂光照条件下实现准确颜色提取。结合 HSV 转换、色度操控和二值化处理，模块实现鲁棒的色块分离，为后续形状和位置识别提供高质量输入。

对于 ADD_VALUE 参数，其取值来源于对色彩的直方图统计，通过颜色统计直方图或者颜色行波形状图，寻找颜色分布的区间，从而确定颜色转换时的阈值，以人民币的颜色 H 值统计直方图为例，如下：

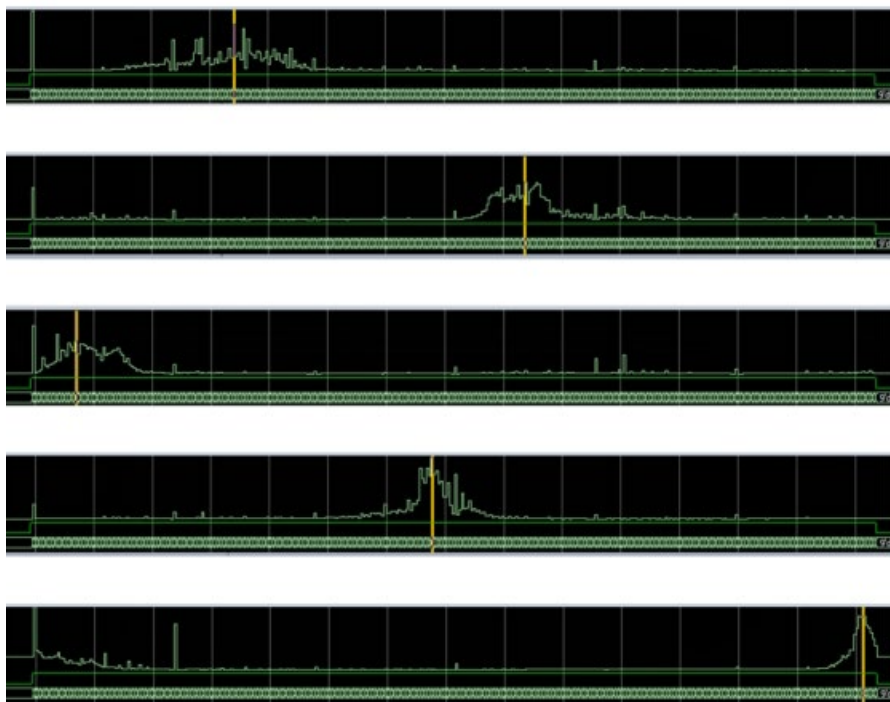


图 3.5.7 人民币颜色 H 分布

在颜色识别模块中，目标颜色的提取依赖于不同颜色在 HSV 色彩空间中的直方图分布特性。通过分析波形特点，可以发现不同颜色在 H 分量上的分布范围存在显著差异，一旦确定了目标颜色的分布范围，便可通过设置特定的阈值参数实现颜色的精确分离。

利用 ModelSim 仿真工具对图像数据对应行的通道波形进行分析。观察目标区域的颜色通道波形，可以直观地确定颜色分布的阈值范围。图 3.5.8 展示了 HSV 转化后二值化阈值参数确定的仿真结果。该图呈现目标区域内不同颜色通道的分布特性，其中横轴表示图像行像素点，纵轴表示 H 值或灰度值的大小，波形峰值和谷值反映颜色的分布区间。

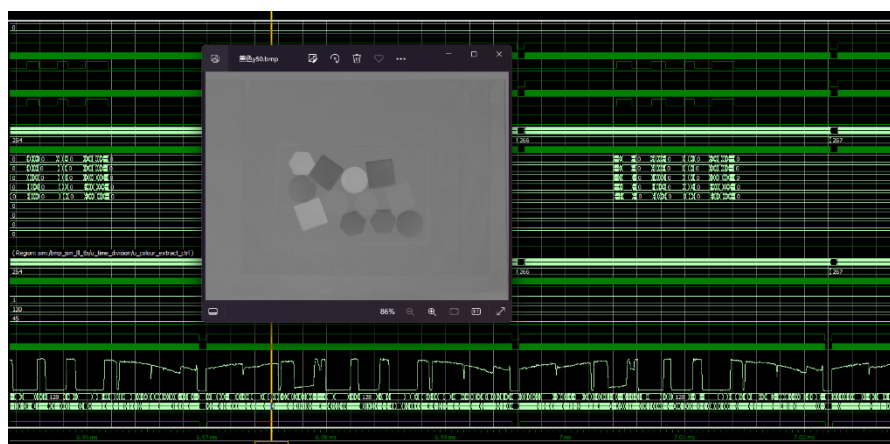


图 3.5.8 确定二值化阈值

3.6 物体形状识别

物体形状识别是本系统视觉处理模块的重要环节，旨在从 OV5640 双目摄像头采集的 720P 分辨率二值化图像中精确识别托盘内 12 个随机摆放物体的形状（圆形、正方形、六边形），为机械臂的分拣任务提供关键数据支持。原始的图片是一副包含 12 个色块的图片，通过合适的灰度二值化阈值设置，可以得到如下图片：

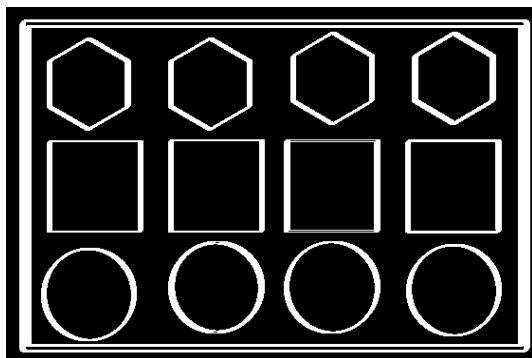


图 3.6.1 二值化图

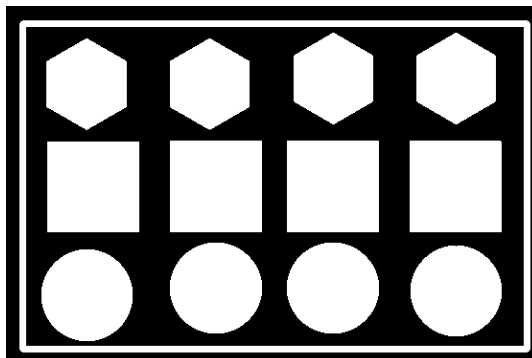


图 3.6.2 Sobel 边缘检测图

可以看出，此时的二值化图片中，包括边框在内的 12 个图片全部呈现白色，在 RGB8 的像素数值上表现为黑色 0，白色 255，二值化处理后表现为黑色 0，白色 1，这样我们就得到了一副由 0 和 1 组成的图片，或者说是一个二维数组，现在只需要分析这个二维数组的特征就可以得到有薄片和没有薄片位置的信息。现在将上面二值化的图像进行水平和竖直投影，对图片进行边缘检测。

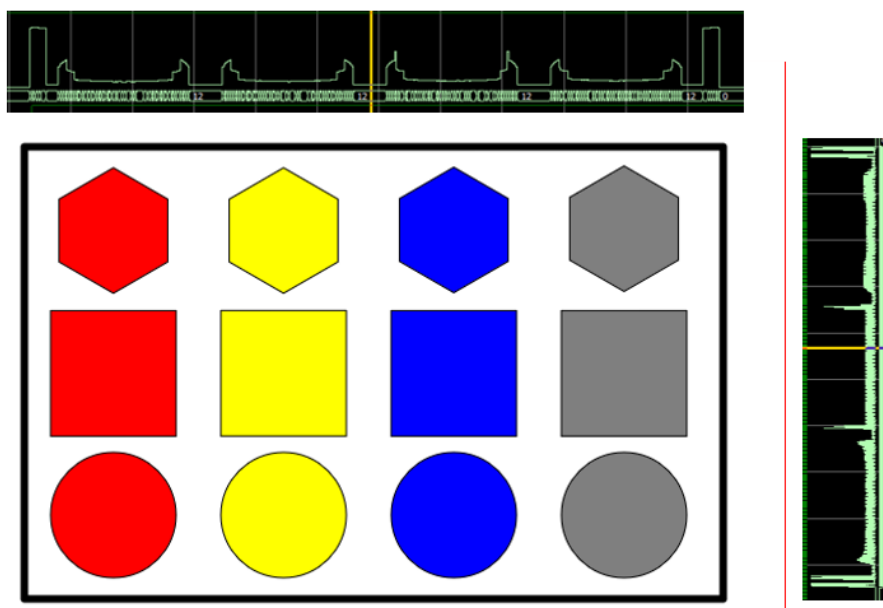


图 3.6.3 水平竖直投影示意

基于投影波形的特征分析，形状识别模块通过寻找波峰和波谷的位置，将图像分割成 12 个小的部分，如图 3-6-4 所示（见附图“图 3-6-4：分割后的色块区域”）。每个分割区域的边界坐标（最左、最右、最上、最下）被提取，用于计算色块的几何中心和形状参数。

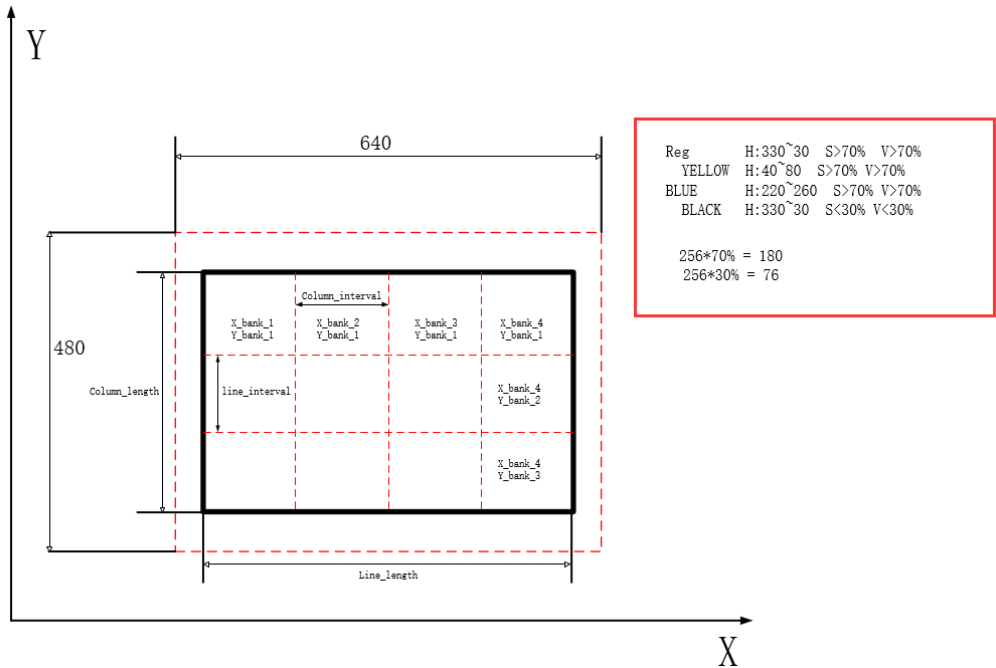


图 3.6.4 托盘分割示意图

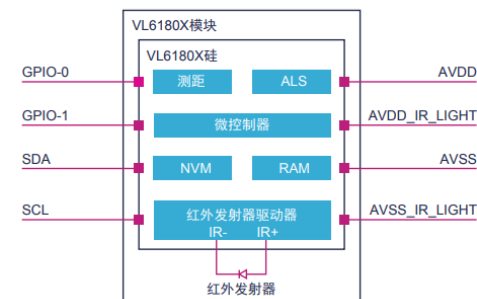
在初步分割出 12 个独立色块区域后，系统进一步采用垂直投影方法对每个物体的边界区域进行详细分析，以实现形状的精确识别。投影过程通过累加二值化图像中目标区域内每一列的像素值，生成相应的直方图，统计像素分布的特性。例如正方形由于其规则的垂直和水平边缘，在投影直方图中呈现较高的峰值，而圆形和六边形因边缘曲率的差异，其投影分布范围呈现不同的特征分布模式。

为进一步提升识别精度，系统结合面积计算和边界框的宽高比特征，对投影分析结果进行多维度验证。面积计算通过统计二值化区域内白色像素（值为 1）的总数，辅助判断物体的整体尺寸，而边界框的宽高比则通过提取外接矩形的最左、最右、最上、最下坐标计算获得。这种多特征融合方法能够校正单一投影分析可能出现的误判。例如，正方形的宽高比接近 1，表明其边界框呈正方形分布，而六边形由于其几何结构的非正交性，宽高比通常略有偏差（例如 0.8 至 1.2 之间）。通过宽高比的校正规则，系统能够更准确地分类圆形、正方形和六边形，避免因边缘噪声或投影偏差导致的错误识别。

3.7 VL6180X 距离传感器

3.7.1 传感器介绍

系统框图



飞行时间原理

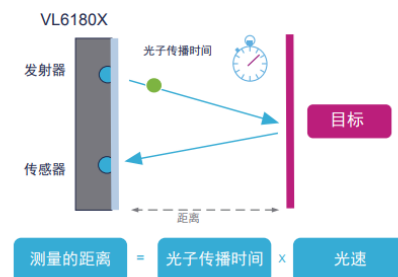


图 3.7.1 传感器介绍

VL6180X 是一种高性能时间飞行 (Time-of-Flight, ToF) 测距传感器，集成了测距传感器，能够以 1 毫米的分辨率测量 0 至 10 厘米的距离。该传感器在较小测距范围内表现出较高的精度，非常适合安装在 Delta 机械臂末端，用于检测末端吸盘与托盘上物体之间的距离。

在本设计中，VL6180X 与紫光同创 PGL22G 板卡通过自编写的软核 I²C 接口实现通信，旨在配置传感器工作模式并实时采集距离数据。设计过程分为以下几个关键步骤：首先，开发软核 I²C 模块，利用 Verilog 语言实现主设备 (FPGA) 与从设备 (VL6180X) 的通信协议。模块包括状态机控制、数据发送/接收逻辑及时钟生成单元，确保 I²C 信号的准确同步。初始化阶段，按照传感器数据手册建议的序列，通过写入特定寄存器配置 VL6180X 为单次测距模式，启用红外激光发射并设置采样间隔。

3.7.2 软核 I2C 原理

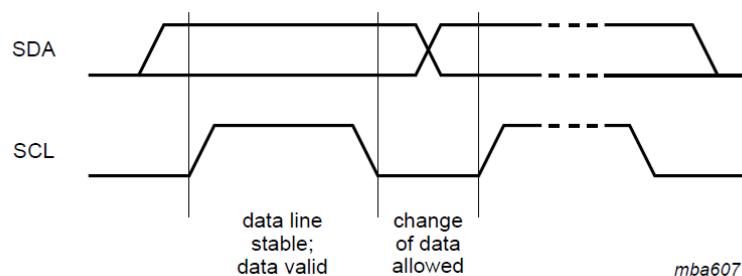


图 3.7.2 数据有效时序图

I2C 只有两条线，一条数据线 SDA，一条时钟线 SCL。SDA 线上的数据在时钟的高电平周期内必须稳定，数据线的高电平或低电平状态只能在 SCL 线上的时钟信号为低电平时才能改变，每传输一个数据位都会产生一个脉冲。

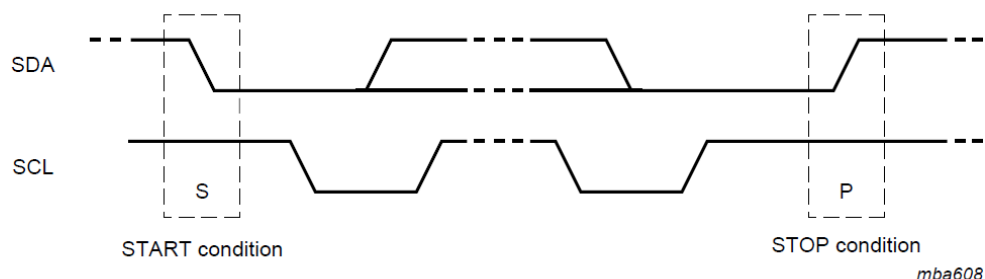


图 3.7.3 开始和停止时序图

当 SDA 由高电平转为低电平，并且 SCL 为高电平时，为**开始条件**。SDA 由低电平转为高电平，并且 SCL 为高电平时，为**结束条件**。

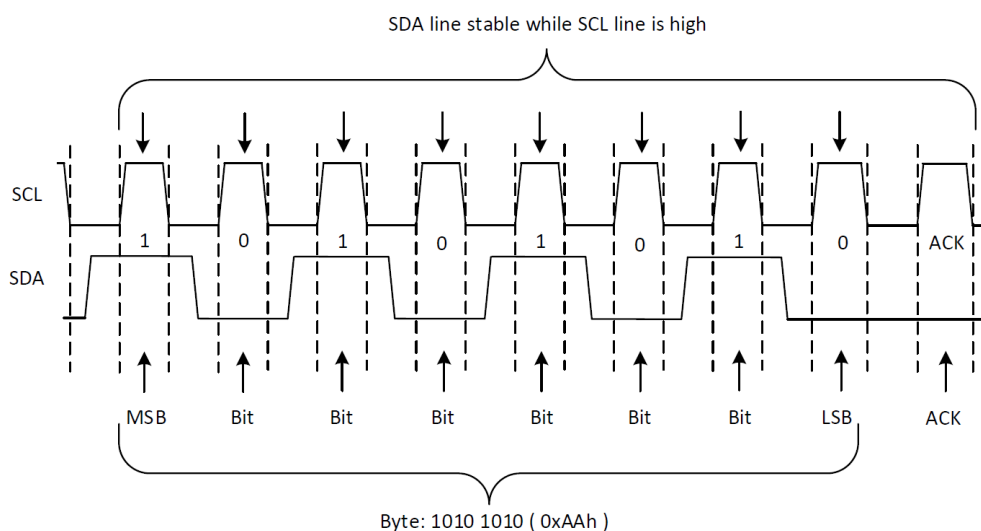


图 3.7.4 数据传输过程时序图

在 SCL 的每个时钟脉冲期间传输一个数据位。一个字节由 SDA 线上的八个 bit 组成。一个字节的地址可以传输的数据是设备地址、寄存器地址或从从机写入或读取的数据。

首先以最高有效位（MSB）传输数据。在 START 和 STOP 条件之间，可以将任意数量的字节数据从主设备传输到从设备。SDA 线路上的数据在时钟周期的高电平阶段必须保持稳定，因为 SCL 高电平时数据线的变化被解释为控制命令。

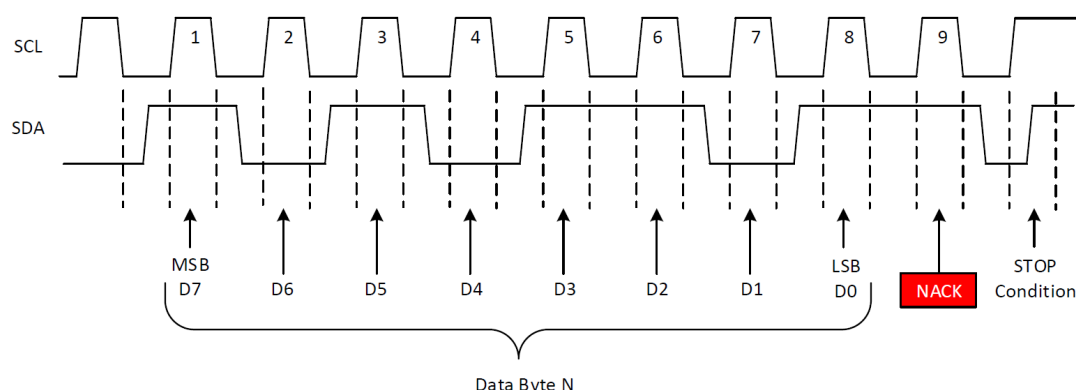


图 3.7.5 ACK 应答和 NACK 不应达时序图

数据的每个字节（包括地址字节）后跟来自接收器的一个 ACK 位。ACK 位允许接收器与发送器通信该字节已成功接收并且可以发送另一个字节。发送应答时，ACK=0（因为都接上拉电阻，因此若 ACK 设置为 1 时，无法判别是否进行了应答操作）。在接收器可以发送 ACK 之前，发射器必须释放 SDA 线路。要发送 ACK 位，接收器应在 ACK/NACK 相关时钟周期（周期 9）的低电平阶段拉下 SDA 线，以使 SDA 线在 ACK/NACK 相关时钟周期的高电平期间稳定为低电平。

2.7.3 软核 I2C 设计

本设计的 I²C 通信协议实现参考了正点原子的例程，并结合与 VL6180X 传感器通信的具体需求进行了针对性优化，以确保通信的正确性、稳定性和高效性。设计中定义了以下关键通信参数：VL6180X 传感器的默认 I²C 设备地址为 7 位二进制值 0101001；FPGA 的系统时钟频率（SYS_CLK_FREQ）设定为 50 MHz；SCL 时钟频率（SCL_FREQ）配置为 250 kHz。

在代码实现上，软核 I²C 模块采用了一个包含 16 个状态的有限状态机设计，全面覆盖 I²C 通信协议的各个阶段。这些状态包括空闲（IDLE）、起始信号 1（START_1）、发送设备地址（SEND_D_ADDR）、应答检测 1（ACK_1）、起始信号 2（START_2）、发送读地址（SEND_RD_ADDR）、读取数据（RD_DATA）、非应答信号（N_ACK）和停止信号（STOP）等。

写操作流程包括：通过 SEND_D_ADDR 状态发送设备地址，随后在 SEND_B_ADDR_H 和 SEND_B_ADDR_L 状态分别发送 16 位寄存器地址的高字节和低字节，紧接着通过 WR_DATA 状态发送数据，并在 ACK_x 状态检测从设备的应答信号。

读操作流程则涉及 START_2 和 SEND_RD_ADDR 状态以重新发送设备地址（切换至读模式），通过 RD_DATA 状态读取数据，在 N_ACK 状态发送非应答信号，并最终在 STOP 状态完成通信。

状态切换由计数器 cnt_i2c_clk（控制 SCL 时钟周期）和 cnt_bit（管理数据位传输）精确控制，i2c_end 信号在 STOP 状态置高，标志着一轮 I²C 通信的结束。这一设计通过 FSM 的精细管理，确保了通信过程的可靠性和时序准确性。

3.7.4 传感器寄存器配置

```

else if(state == S_DATA)
    case (state_data)
        SD_READ0 : begin i2c_addr = 16'h004d; i2c_wrddata = 8'h00; end
        SD_WRITE0: begin i2c_addr = 16'h0018; i2c_wrddata = 8'h01; end
        SD_READ1 : begin i2c_addr = 16'h004f; i2c_wrddata = 8'h00; end
        SD_READD : begin i2c_addr = 16'h0062; i2c_wrddata = 8'h00; end
        SD_WRITE1: begin i2c_addr = 16'h0015; i2c_wrddata = 8'h07; end
        default  : begin i2c_addr = 16'h0000; i2c_wrddata = 8'h00; end
    endcase
else
    begin i2c_addr = 16'h0000; i2c_wrddata = 8'h00; end

```

图 3.7.6 传感器寄存器数据采集

基于前述的 i2c_ctrl 软核，设计开发了 vl6180x 模块，专门负责 VL6180X 传感器的初始化配置和实时数据采集功能，为 Delta 机械臂的高精度定位提供支持。

顶层状态机（state）包含两个状态：初始化状态（S_INIT）和数据采集状态（S_DATA）。在 S_DATA 状态下，定义 6 个子状态（state_data）：读取状态寄存器（SD_READ0）、启动测量（SD_WRITE0）、检查测量完成（SD_READ1）、读取距离数据（SD_READD）、清除中断（SD_WRITE1）和等待（SD_WAIT），覆盖测距任务完整流程。

在初始化阶段，vl6180x 模块通过计数器 init_cnt 执行 39 次 I²C 写操作（INIT_NUM=6'd39），配置 VL6180X 寄存器，遵循数据手册推荐流程。例如，写入 0x0207=0x01 启用测距功能，0x0208=0x01 设置默认工作模式，并优化 0x00e3 至 0x0014 等寄存器，初始化过程确保传感器稳定运行。

数据采集阶段，模块在 S_DATA 状态下循环执行以下流程：读取 0x004d 确认传感器状态；写入 0x0018=0x01 启动单次测量；读取 0x004f（检查位 2）确认测量完成；从 0x0062 读取 8 位距离值（单位毫米）；写入 0x0015=0x07 清除中断。为控制测量间隔，模块使用计数器 WAIT_COUNT=16'd100000（50 MHz 时钟，约 50 ms）实现 SD_WAIT 状态等待。

3.8 HDMI 显示

HDMI（High-Definition Multimedia Interface）作为音视频设备互联的标准，依托 TMDS（Transition Minimized Differential Signaling）技术实现高速、可靠的数据传输。本节阐述 HDMI 硬件设计与视频时序标准的实现。

3.8.1 HDMI 硬件

本设计基于开发板 HDMI 接口，采用四对 TMDS 差分信号通道：一对传输时钟信号，三对负责数据传输。HDMI 传输原理与 DVI 一致，均依赖 TMDS 技术。在发送端，HDMI 接口接收 24 位并行 RGB 数据（每像素 R、G、B 各 8 位），经编码和并/串转换后分配至三个数据通道，同步时钟信号通过专用通道发送。接收端对串行信号解码并进行串/并转换，恢复为并行数据，结合时钟信号实现精准同步后送至显示控制单元。

TMDS 编码是高效传输的核心。每个数据通道将 8 位视频或音频数据编码为 10 位序列，通过异或/异或非逻辑运算生成前 8 位，第 9 位指示运算方式，第 10 位确保直流平衡。此编码机制优化传输最小化和抗干扰能力，支持视频数据（8 位）、控制数据（2 位）或数据包（4 位，含音频和纠错码等）。HDMI 传输分为三个阶段：视频数据周期（传输像素数据）、控制数据周期（传输同步信号等）和数据岛周期（传输音频和附加信息），满足多样化传输需求。

3.8.2 视频时序标准

HDMI 显示采用逐行扫描，从屏幕左上角逐点由左至右扫描，完成一行后通过消隐信号抑制电子束，返回下一行起点。行同步信号确保每行扫描同步。一帧扫描完成所有行后，场同步信号触发扫描返回左上角，进行场消隐，开始下一帧。一行扫描的持续时间称为水平扫描时间，其倒数为行频率；一帧扫描的持续时间称为垂直扫描时间，其倒数为场频率（即屏幕刷新率），常见值为 60 Hz、75 Hz 等。本设计采用标准的 60 Hz 场频率。



3.8.1 行同步时序

3.8.3 HDMI 输出程序设计

HDMI 输出模块在本项目中负责处理双目摄像头采集的视频流，并将其高效传输至外部 HDMI 显示设备，以实现实时视频监控和识别结果的可视化。为满足工业场景中对低延迟和高稳定性的要求，系统采用了一系列数据处理与存储机制，确保视频流的连续性和一致性。双目摄像头采集的 720P 分辨率 RGB565 格式视频数据首先存储至写 FIFO（First-In-First-Out）缓冲区，FIFO 的深度和宽度根据数据吞吐量设计，缓冲区能够暂时缓存摄像头输出的突发数据，防止因数据采集速率与处理速率不匹配导致的丢帧问题。随后的数据处理通过生成 DDR3 的 Burst 写请求，将 FIFO 中的数据写入 DDR3 存储器，Burst 模式的连续读写操作优化了内存带宽利用率，提升了数据传输效率。

读取过程与写入过程类似，当读 FIFO 中的数据量低于预设阈值时，系统触发 DDR3 的 Burst 读请求，从存储器中提取视频数据以供 HDMI 控制器处理。DDR3 作为中间存储介质，提供了足够的容量和带宽，支持双路视频流的并行处理。为实现双目摄像头的同步显示，设计中实例化了两个 frame_read_write 模块，分别负责管理左目和右目视频数据的存储与读取操作。每路视频数据在 DDR3 中的存储地址被分配为独立区域，通过地址偏移量避免数据冲突，确保高效访问和数据完整性。左目数据用于显示二值化后的处理结果，右目数据则叠加识别后的物体画框标注，两种画面通过 HDMI 控制器在显示屏上划分为左右两部分，实时呈现。

```
.read_finish      (\
.read_addr_0      (25' d0                                ), //The first frame address is 0
.read_addr_1      (25' d2073600                           ), //The second frame address is 25' d207
.read_addr_2      (25' d4147200                           ),
.read_addr_3      (25' d6220800                           ),
.read_addr_index   (ch0_read_addr_index                   ),
.read_len          (25' d196608                           ),//frame size 1024 * 768 * 16 / 64
```

3.8.2 第一路视频存储地址

```
.read_finish      (\
.read_addr_0      (25' d8294400                           ), //The first frame address is 0
.read_addr_1      (25' d10368000                          ), //The second frame address is 25' d207
.read_addr_2      (25' d12441600                          ),
.read_addr_3      (25' d14515200                          ),
.read_addr_index   (ch1_read_addr_index                   ),
.read_len          (25' d196608                           ),//frame size 1024 * 768 * 16 / 64
... ..            (\
```

3.8.3 第二路视频存储地址

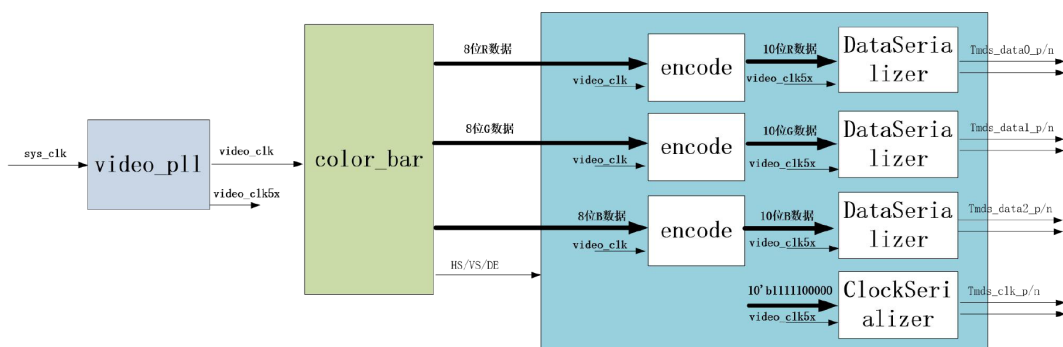


图 3.8.4 实现逻辑框图

HDMI 输出显示模块通过三个核心模块实现视频数据的处理与传输，结构如图 3.8.4 所示，确保双目摄像头采集的 720P RGB565 视频流实时输出至显示设备。vidio_pll 模块生成稳定的时钟信号，为视频传输和 TMDS 编码提供精确时序基准。基于 $1024 \times 768 @ 59.94 \text{ Hz}$ 时序设计（参考 3.8.2），vidio_pll 利用锁相环技术将 FPGA 50 MHz 系统时钟调整至约 65 MHz 点时钟频率，保证信号同步与稳定传输。dvi_encoder 模块负责将 VGA 格式视频信号转换为符合 HDMI 1.4 协议的 DVI 信号，完成 TMDS 差分编码和数据封装。视频流处理模块作为核心数据处理单元，从 DDR3 存储器读取双目摄像头 720P RGB565 视频数据，并整合图像处理算法的输出。整合后的视频流经 dvi_encoder 处理，通过 HDMI 接口输出，实现双路视频的实时显示。

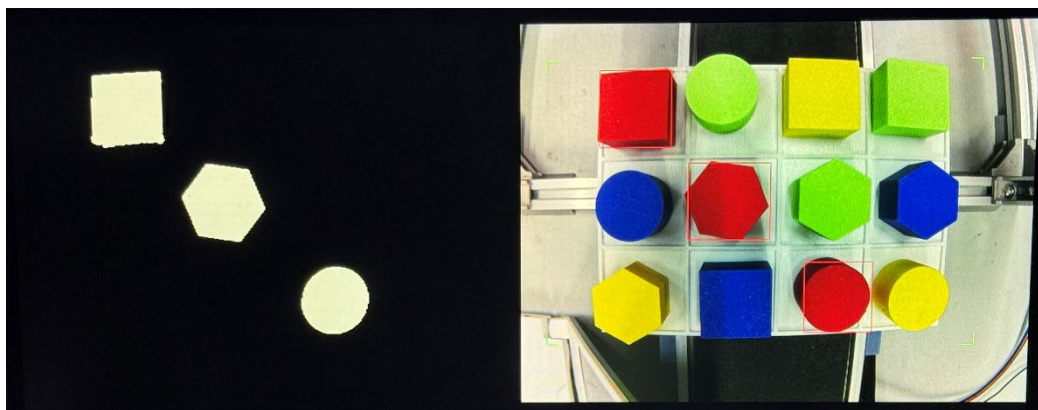


图 3.8.5 HDMI 显示画面

为实现双路视频的同步显示，设计中调用了两个 video_rect_read_data 模块，将显示屏幕划分为左右两部分，分别呈现来自双目摄像头的画面。左边显示左边摄像头的画面，数据经由图像处理的算法，显示二值化之后的画面；右边显示右边摄像头的画面，在原始画面上对识别的物体画框。

直观的双画面呈现显著提升了人机交互体验，为操作人员提供实时监控和调试支持。

3.9 上位机交互

上位机系统采用模块化设计，基于 Qt 框架构建，包含输入模块、计算模块、通信模块和显示模块。输入模块负责接收用户输入的三维坐标或关节角度数据；计算模块实现运动学正逆解算法，完成坐标与角度的转换；通信模块通过串口与 Delta 机械臂硬件交互；显示模块负责实时可视化运动轨迹和状态信息。界面采用深色调设计，配以蓝色边框和紫色按钮，增强用户体验。

3.9.1 上位机界面

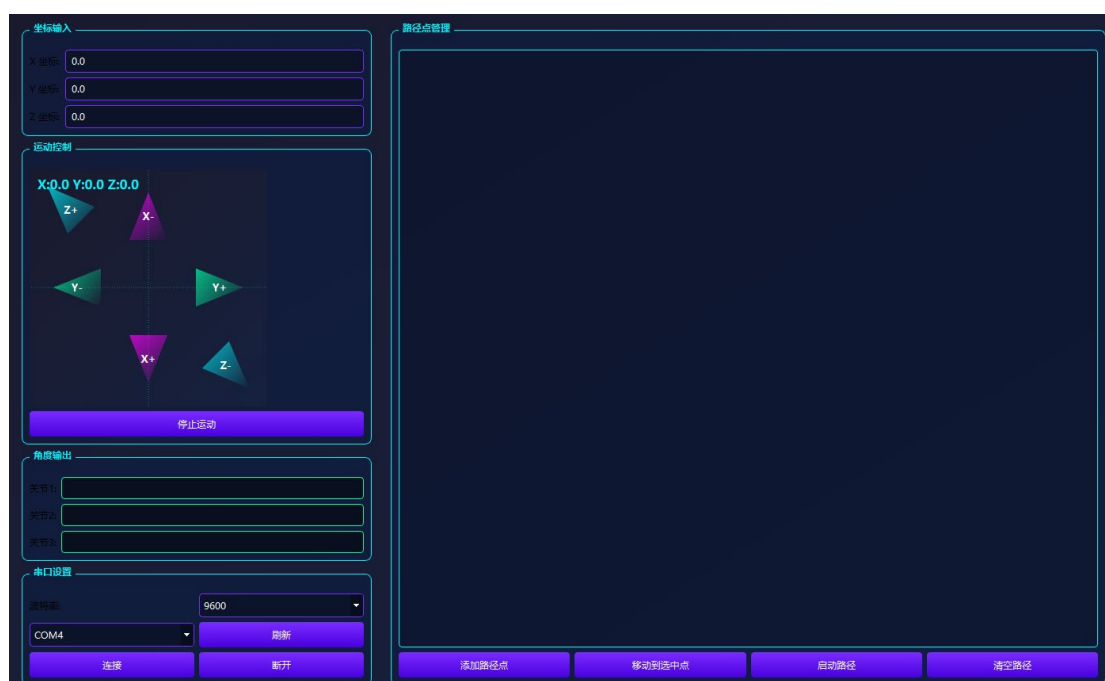


图 3.9.1 上位机界面

(1) 串口设置

在上位机界面的左下角，设计了一个简洁且功能齐全的串口设置区域，作为用户与机械臂控制系统交互的核心入口。这一区域通过直观的布局和高效率的操作方式，为操作人员提供了与紫光同创 PGL22G FPGA 板卡通信的便捷手段。

设计中，串口设置区域支持串口波特率的调节和串口接口的选择，用户可以通过下拉菜单轻松选择目标串口，例如常见的 COM4，并根据需求调整波特率至 115200 bps，匹配 FPGA 通信模块的默认设置。这种灵活的配置选项确保了上位机与硬件之间的兼容性，适应不同的硬件环境和通信需求。

（2）坐标输入

在上位机界面的左上角是用户输入机械臂末端的空间坐标点的入口，可分别设置 x 、 y 、 z 的坐标，用于直观的展示。当输入 x 、 y 、 z 坐标值后，上位机内部会有 Delta 机械臂的运动学逆解算法，能够实时输出对应的三个关节角度值，让机械臂的运动控制更为直观高效。角度输出的 UI 界面如图 3-3 所示：



图 3.9.2 坐标输入与坐标输出

（3）交互控制

系统设计了直观的 3D 交互控制界面，支持多种操作方式以满足用户对机械臂运动的灵活控制需求。界面提供 6 个方向按键，分别控制 X 、 Y 、 Z 坐标的正负方向调整，用户可通过点击进行微调，或长按实现快速调整，操作便捷高效。界面的右侧为大型可视化窗口，实时展示机械臂的三维运动轨迹和坐标点，增强用户对运动状态的直观感知。交互控制功能包括以下核心操作：

添加路径点：用户在坐标输入框中输入目标位置的 X 、 Y 、 Z 值后，点击“添加路径点”按钮，系统会将该坐标点保存至路径点管理列表，并实时显示已添加的路径点数量（如“5 个路径点”），便于用户查看和管理。

移动至指定路径点：用户可在路径点管理列表中选中某一目标点，点击“移动至选中点”按钮，系统通过串口将该点坐标发送至下位机，驱动机械臂自动运动至目标位置。

自动执行路径：点击“启动路径”按钮，系统按顺序读取路径点管理列表中的所有坐标，依次发送至下位机，机械臂将按照预设路径依次运动，实现自动化轨迹控制。

清空路径点：当需要重新规划路径时，用户可点击“清空路径”按钮，一键清除路径点管理列表中的所有数据，确保新路径规划的准确性。该功能通过直观的路径点管理界面，支持单点定位和连续轨迹控制，为用户提供了灵活高效的机械臂运动路径调整方式。

3.9.2 功能实现

(1) 基本的串口通信

串口通信通过 C++ 后端逻辑与 QML 前端界面的协同实现，确保高效、稳定。C++ 后端利用 Qt 的 `QSerialPort` 库，通过串口管理类封装串口初始化、参数配置（波特率 9600）、状态管理及数据收发功能。借助 Qt 信号与槽机制，实时监控串口状态（如连接或断开）及接收数据，触发相应槽函数更新状态或处理数据，确保通信可靠性。

QML 前端提供直观操作界面，包含“打开串口”“关闭串口”“刷新串口”等按钮，通过与 C++ 后端的交互实现操作。串口配置（如波特率、端口）经 QML 传递至后端，支持动态调整。QML 与 C++ 交互基于上下文属性和信号与槽机制，串口管理类作为上下文属性暴露给 QML，控件可直接调用后端方法（如发送数据）。后端状态变化或接收数据通过信号传递至 QML，槽函数实时更新界面状态显示框，便于用户监控。

(2) 指令发送

指令发送功能是上位机与 Delta 机械臂交互的核心环节。系统通过串口将用户输入的坐标或路径点数据转换为控制指令，并发送至下位机驱动机械臂运动。

在 C++ 后端，指令发送基于 `QSerialPort` 的数据写入功能实现。系统首先将用户输入的坐标数据（X、Y、Z）通过运动学逆解算法转换为关节角度值，然后按照预定义的通信协议将角度值封装为指令数据包。指令数据包包含起始位、数据位、校验位和结束位，确保数据传输的完整性。串口管理类调用 `QSerialPort` 的 `write()` 方法将数据包发送至下位机，同时通过 `read()` 方法接收下位机的反馈数据，验证指令是否成功执行。

在路径点自动执行模式下，系统会按顺序遍历路径点管理列表中的坐标点，逐一转换为指令并发送至上位机。为避免指令发送过于频繁导致机械臂运动不稳定，系统设计了时间间隔控制机制，确保每次指令发送后等待机械臂完成当前动作，再发送下一指令。

在 QML 前端，用户通过“启动路径”按钮触发指令发送流程。界面会实时显示当前执行的路径点序号和机械臂状态，增强用户对执行过程的感知。若通信过程中发生错误（如串口断开或数据校验失败），系统会通过界面弹窗提示用户，并暂停指令发送，待用户重新连接串口后恢复操作。

4 性能指标分析

4.1 机械臂运动控制

机械臂运动控制是本系统性能的核心组成部分，其性能直接决定了分拣任务的成功率和效率。

步进电机的稳定性提供了对整套系统提供了强有力的支持，配合我们开发的梯度加减速算法，使稳定性得到进一步的提升。在整套系统中，Delta 机械臂使用了 3 个步进电机，平行四边形连杆机械臂使用了 4 个步进电机，我们对每一个电机的加速加速度、减速加速度、最大速度、最小速度都做了对应的适配。

具体参数如下表，此参数对应的芯片是紫光 PGL22G，主频 50MHz。

表 4.1 步进电机参数

步进电机序号	最大速度脉冲频率	最小速度脉冲频率	加速/减速加速度
连杆机械臂平移轴	3000	6000	10
连杆机械臂旋转轴	2000	8000	1
连杆机械臂大臂轴	3000	8000	2
连杆机械臂小臂轴	3000	8000	2
Delta 机械臂	5000	10000	100

4.1.1 Delta 机械臂性能

Delta 机械臂作为本系统的主要执行单元，通过门型路径规划实现了从拾取点到放置点的精准移栽操作，其运动精度在测试中达到了 ± 1 毫米的水平，满足工业分拣任务的基本要求。结合前述优化的步进电机控制参数，每次拾取与放置操作的平均耗时约为 2 秒。然而，由于 Delta 机械臂的步进电机减速比仅为 1:3，且其结构件采用 3D 打印材料，整体强度有限，导致机械臂的运动速度受到显著限制，无法实现更高速的操作。这种速度瓶颈主要源于机械结构和驱动系统的设计特性。

在尝试提高步进电机脉冲频率以提升速度时，Delta 机械臂的结构特性带来了新的挑战。由于其并联机构的设计，末端执行器的速度会随着驱动臂运动的加速而成倍放大。当运动方向发生突变(例如门型路径中上升段与水平段的衔接处)时，末端因惯性作用会产生一定的误差位移，导致定位精度的下降。此外，步进

电机的减速通过同步带实现 1:3 的减速比，尽管系统中加入了弹簧对同步带进行张紧以减少松弛，但当运动速度进一步提高时，同步带仍会出现打滑现象。这种打滑不仅加剧了误差位移，还显著降低了机械臂末端的重复定位精度。测试表明，在高速运动下，末端定位误差可达 ± 5 毫米以上，远超分拣任务的精度要求。

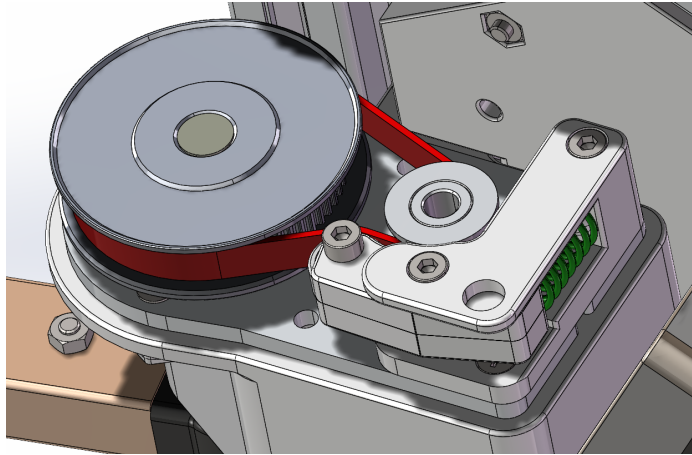


图 4.1.1 Delta 机械臂减速结构

在 4.1.1 图中，红色的是 2GT 同步带，绿色的是皮带紧张弹簧，小齿轮有 20 齿连接步进电机，大齿轮有 60 齿连接主动臂，通过皮带实现 1:3 的减速比。

为解决上述问题，我们通过多次实验优化了步进电机的运行参数，最终在保证重复定位精度的前提下，确定了最优速度设置。表 4-1 列出了经过反复测试得出的参数配置。在该参数配置下，机械臂能够在 ± 1 毫米的定位精度范围内稳定运行，同时将拾取与放置的单次操作耗时控制在 2 秒左右。这一速度虽然无法进一步提升，但通过梯度加减速算法和路径规划优化，显著减少了末端振动和同步带打滑现象，为系统的整体稳定性和可靠性提供了保障。

在系统的协同操作中，Delta 机械臂和平行四边形连杆机械臂的分工明确且高效。Delta 机械臂负责从托盘中拾取物体并按颜色分类放置，每完成三次拾取与放置操作后，系统会触发平行四边形连杆机械臂的运动，开始执行托盘的拿取和更换任务。与此同时，Delta 机械臂利用这一时间窗口执行复位操作，通过限位开关将机械臂精确地归位至零点状态。这一复位机制通过硬件限位检测，确保机械臂回到初始位置，从而有效消除因多次操作而累积的定位误差。测试表明，复位操作耗时约 1 秒，配合限位开关的精度 (± 0.2 毫米)，能够将累计误差清零，使 Delta 机械臂在下一轮拾取任务中恢复至 ± 1 毫米的定位精度。这种协同设计不仅优化了机械臂的工作节奏，还显著提升了系统在长时间运行中的稳定性和可靠性，为分拣任务的连续性提供了保障。

```

RESETTING: begin
    // 检查限位开关, 停止对应电机
    if (!limit_switch_1)
        motor1_active <= 1'b0;
    if (!limit_switch_2)
        motor2_active <= 1'b0;
    if (!limit_switch_3)
        motor3_active <= 1'b0;
    // 所有电机停止时进入完成状态
    if (!motor1_active && !motor2_active && !motor3_active)
        state <= FINISH;
end

```

图 4.1.2 Delta 机械臂复位操作代码

4.1.2 平行四边形连杆机械臂性能

相较于 Delta 机械臂的高精度要求, 平行四边形连杆机械臂在本系统中主要承担搬运和放置托盘的任务, 其精度需求相对较低。托盘的拿取和放置采用磁吸方式, 与 Delta 机械臂吸取物体的操作相比, 磁吸机制显著简化了抓取和释放的复杂度, 使得连杆机械臂在执行任务时更易于实现稳定的拿起与放下操作。磁吸方式通过两个电磁铁的通断控制实现, 吸力可达 6 牛顿以上, 足以应对托盘的重量 (约 200 克), 同时避免了机械夹爪可能出现的夹持不稳问题。

基于任务特点, 我们在连杆机械臂上并未引入过多的复杂算法, 仅保留了步进电机梯度加减速算法和逆运动学解算模块, 以满足基本的运动控制需求。步进电机梯度加减速算法通过动态调整脉冲频率, 确保机械臂在启动和停止阶段的平滑过渡, 减少因速度突变引发的振动, 单次托盘搬运耗时约为 4 秒, 定位误差控制在 ± 2 毫米以内。尽管连杆机械臂的精度要求较低, 但其扩展的 Y 轴运动范围 (-300 毫米至 700 毫米) 通过平移轴步进电机的引入得以实现, 显著提升了机械臂的操作灵活性, 确保其能够适应传送带上不同位置的托盘搬运需求。整体设计在简化算法复杂度的同时, 依然满足了系统对托盘搬运任务的稳定性与效率要求。

4.1.3 逆运动学解算

逆运动学解算是本系统核心模块, 扮演着承上启下的关键角色, 其主要功能是将目标坐标转换为机械臂各关节的电机角度, 确保 Delta 机械臂和平行四边形连杆机械臂的末端执行器能够精确到达预定位置。这一模块的计算精度和速度直接影响系统的整体性能, 尤其在高速连续运动场景中, 逆运动学解算的速度成为影响机械臂响应时间的重要因素。为评估其性能, 我们利用 ModelSim 仿真工具对逆运动学解算模块进行了详细测试和优化。

(1) 解算精度

为确保在 FPGA 中进行逆运动学解算时不因定点运算而丢失精度，设计中对数据进行了多级放大处理。以 Delta 机械臂为例，输入的空间坐标 (X, Y, Z) 在进入逆运动学解算模块前被直接放大 16 倍，以提高运算的初始精度。随后，在数据预处理模块 `pretreat_data` 中，中间计算参数被进一步放大 65536 倍，以满足复杂公式对精度的需求。在角度计算模块 `calculate_angle` 中，输出的电机角度值也被放大 256 倍。最终，步进电机控制模块接收这一放大了 256 倍的角度值，并在生成脉冲数时通过右移 8 位还原为真实角度数据。所有放大倍数均设计为 2 的次方形式，使 FPGA 能够通过高效的移位操作完成放大与还原，极大节省了紫光同创 PGL22G 板卡的运算资源，同时避免了浮点运算带来的硬件开销。

为验证 FPGA 实现的逆运动学解算精度，我们将 Verilog 编写的逆运动学解算模块与 Python 实现的浮点运算版本进行了对比分析。

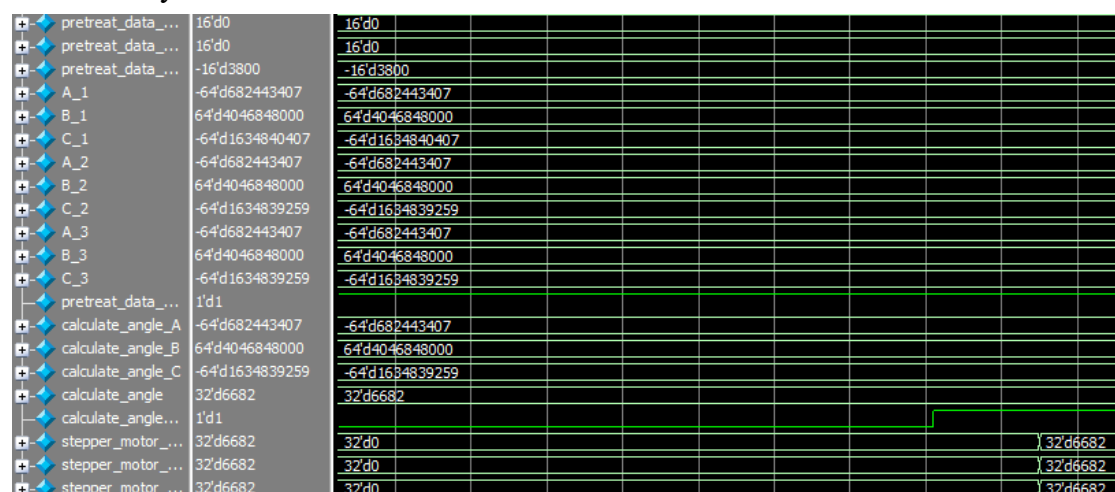


图 4.1.3 ModelSim 仿真 Delta 机械臂逆运动学解算

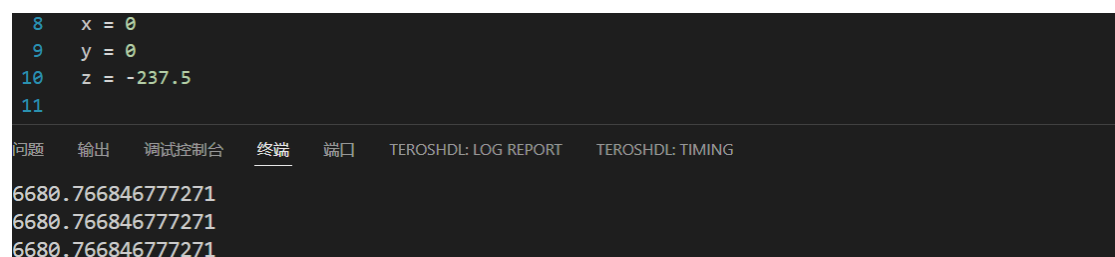


图 4.1.4 Python 逆解结果

为进一步验证 Verilog 实现的逆运动学解算模块的精度，我们针对 Delta 机械臂输入一组特定坐标 (0, 0, -3800) 进行了测试，其中输入坐标已放大 16 倍。在 FPGA 的 Verilog 实现中，角度计算结果如图 4.1.3 所示，输出值为 6682，该值对应于放大 256 倍后的电机角度值。还原为真实角度后，计算为 $6682 \div 256 =$

26.1015625°。相比之下，Python 实现的浮点运算程序计算得到的结果如图 4.1.4 所示，放大 256 倍后的值为 6680.77，真实角度为 26.09677734375°。

对比两者还原后的真实角度值，差距约为 0.0048°。可以看出，Verilog 编写的逆运动学解算代码在紫光同创 PGL22G FPGA 平台上的精度极高。进一步分析，0.0048°的角度误差折算到 Delta 机械臂末端执行器的位置误差约为 0.02 毫米，远低于系统要求的±1 毫米定位精度。

(2) 解算速度

通过 ModelSim 仿真工具对解算模块进行了详细测试，分析了各子模块的时钟周期消耗和整体解算速度。图 4.1.5 展示了 Delta 机械臂逆运动学解算的仿真波形，在 50 MHz 的主频下，完整解算过程耗时 235 个时钟周期，大约 4.7 微秒。

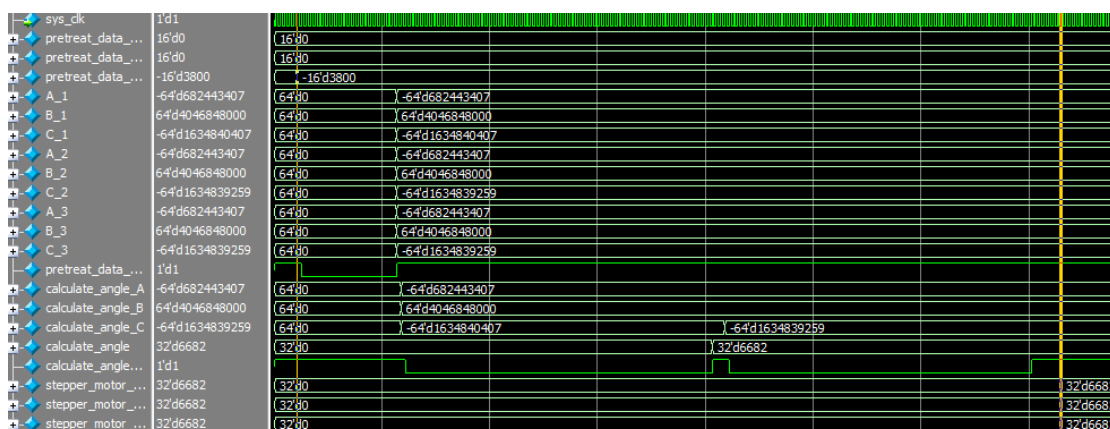


图 4.1.5 Delta 机械臂仿真解算过程

图 4.1.6 展示了 pretreat_data 子模块的仿真波形。对输入坐标进行预处理，需完成 10 个乘法运算。由于板卡资源有限，模块仅使用一个乘法器 IP 核，通过分时复用技术降低资源占用。分时复用过程消耗 20 个时钟周期，整个模块总共耗时 23 个时钟周期。虽然分时复用牺牲了部分速度，但显著减少了 DSP 资源需求，为其他模块的并行计算腾出了空间。

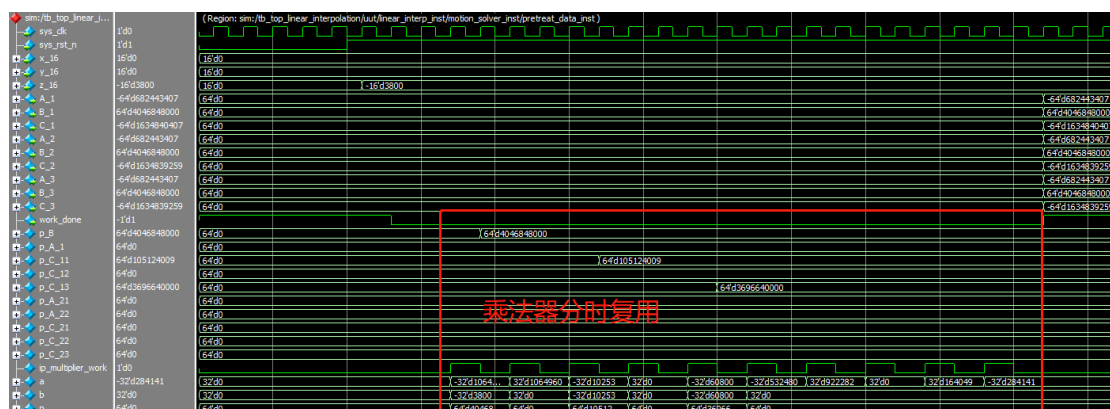


图 4.1.6 pretreat_data 模块仿真波形图

期耗时仅 400ns，对于机械臂控制的实时性要求而言，这一延迟可以忽略不计，同时大幅降低了资源消耗。

通过乘法器分时复用的优化，最终资源配置为：Delta 机械臂逆运动学解算模块使用两个 32 位乘法器，总计消耗 8 个 APM；平行四边形连杆机械臂逆运动学解算模块同样使用两个 32 位乘法器，消耗 8 个 APM。两套模块的总资源占用降至 16 个 APM，远低于 PGL22G 芯片的 30 个 APM 上限。这种资源优化使两个机械臂的逆运动学解算模块能够集成于同一块紫光同创 PGL22G 板卡上，充分利用硬件资源，同时保持解算精度（误差小于 0.01° ）和速度（约 4.7 微秒）。

Logic Utilization	Used	Available	Utilization(%)
Use of ADC	0	1	0
Use of APM	30	30	100
Use of BKCL	3	6	50
Use of CLMA	3027	3274	93
FF	5559	19644	29
LUT	9965	13096	77
LUT-FF pairs	1586	13096	13
Use of CLMS	1024	1110	93
FF	1867	6660	29
LUT	3261	4440	74
LUT-FF pairs	503	4440	12
Distributed RAM	0	4440	0

图 4.1.8 资源消耗报告图

根据上图所示，紫光同创 PGL22G FPGA 板卡在实现双机械臂逆运动学解算模块等功能后，资源利用率呈现出较高的占用状态。报告显示，APM 资源已全部用尽，这一结果源于逆运动学解算模块中乘法运算的高需求，尽管通过分时复用策略将初始的 123 个 APM 需求优化至 16 个，但其他模块在综合网表时也调用了一定的 APM 资源，用于执行简单的乘法或加法运算，以节省 LUT 资源的消耗。这种资源调度策略在一定程度上缓解了逻辑单元的压力。从逻辑资源的角度来看，工程整体使用了 77% 的逻辑单元，其中 CLMA 和 CLMS 的利用率均高达 93%。

当前设计已接近 PGL22G 板卡的硬件极限，尽管如此，通过乘法器分时复用和模块化设计，系统依然在资源约束下实现了预期的功能，逆运动学解算速度（4.7 微秒）和精度（误差 0.0048° ）均满足分拣任务的需求。

4.2 传感器数据处理

4.2.1 颜色与形状识别

在本项目中，所有工程项目的文件均通过 ModelSim 仿真工具进行了全面验证，以确保颜色与形状识别模块的准确性和实时性。图 4.2.1 展示了颜色与形状识别模块的仿真波形，详细记录了从阈值设置到坐标更新的完整过程。仿真结果显示，在 50 MHz 主频下，从给出颜色阈值到更新出单个颜色物体的坐标位置，耗时为 12.6248 毫秒。整个识别过程需依次处理四种颜色（红、黄、蓝、绿），总耗时约为 50.5136 毫秒，与理论计算值一致。

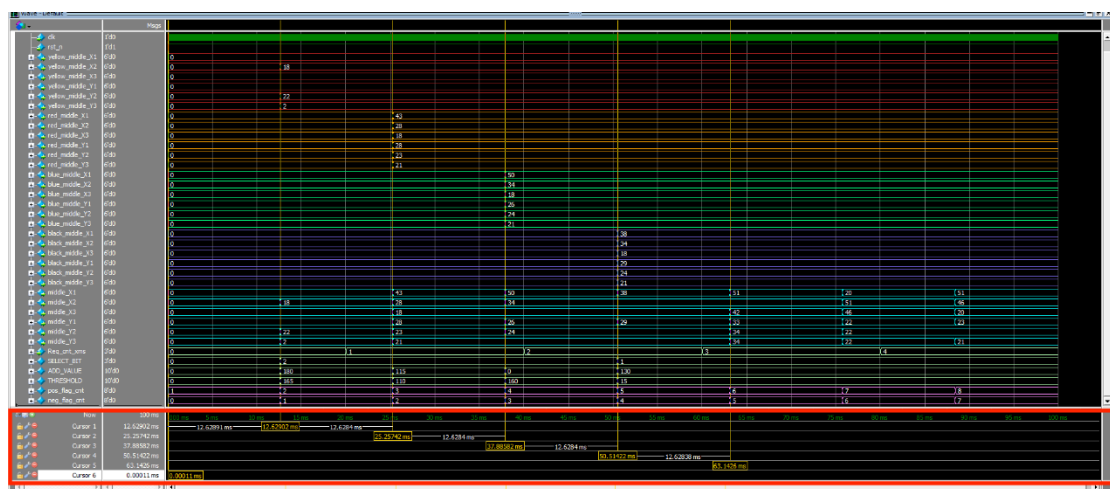


图 4.2.1 识别仿真波形图

波形图中，各颜色物体的坐标更新过程清晰可见：红色波形表示黄色物体的坐标，橙色波形表示红色物体的坐标，蓝绿色波形表示蓝色物体的坐标，紫色波形表示绿色物体的坐标，而蓝色波形则为最终更新出的色块中心坐标点，色块坐标直接从蓝色波形中提取。

图下方，绿色波形表示阈值的变化，用于动态调整颜色提取范围；酒红色波形为场同步信号的上升沿和下降沿计数器，用于同步视频帧处理。识别模块基于 HSV 色彩空间转换和投影分析，首先通过 H 值调整（ADD_VALUE）和灰度通道选择（SELECT_BIT）提取目标颜色，随后利用水平和竖直投影分割色块区域，最后结合面积和宽高比特征完成形状分类（圆形、正方形、六边形）。

ModelSim 仿真结果表明，锁定一组颜色的平均耗时为 12.6248 毫秒，完成全部四种颜色的识别总耗时为 50.5136 毫秒。为满足分拣任务的实时性要求，后续可通过并行处理优化或减少投影计算量，将总耗时压缩至 30 毫秒以内。

4.2.2 物体定位精度

在物体识别的精度优化过程中，由于时间原因，我们对物体定位任务进行了简化。通过将托盘划分为 12 个固定方格，限定物体放置于方格内，避免了随机摆放的复杂性，从而降低了定位算法的实现难度。在这种简化场景下，系统仅需完成物体的形状与颜色识别，即可确定其所在方格位置，为机械臂的分拣任务提供初步定位数据。

然而，随着测试的深入，发现物体为立体结构时，存在一定挑战。当物体不在摄像头正下方时，OV5640 双目摄像头会拍摄到侧面视角，导致计算出的投影面积与实际垂直投影面积产生偏差。这种角度引起的面积失真直接影响形状分类的准确性，特别是在区分圆形、正方形和六边形时，可能导致误判。

为解决这一问题，我们通过反复的测试和 ModelSim 仿真，分析了不同视角下物体的投影面积特性，最终确定了三种形状的面积区别阈值。测试过程中，进行了 12 次随机摆放实验，统计了各形状在不同角度下的投影面积数据，并结合边界框的宽高比进行校正。

表 4.2.1 物体识别准确率

次数	准确识别数	颜色错误数	形状错误数	颜色识别平均准确率	形状识别平均准确率	识别率
1	12	0	0	98.61%	90.97%	89.58%
2	12	0	0			
3	11	0	1			
4	10	0	2			
5	11	0	1			
6	11	0	1			
7	10	0	2			
8	9	1	2			
9	10	0	2			
10	11	0	1			
11	10	1	1			
12	12	0	0			

4.3 人机交互接口

4.3.1 上位机交互

上位机交互模块作为本系统的人机交互核心，为操作人员提供了直观、高效的控制和监控界面，显著提升了 Delta 机械臂和平行四边形连杆机械臂的操作便捷性。该模块基于 Qt 框架开发，充分利用其跨平台特性和强大的图形界面支持，运行于 PC 端。界面设计采用 QML 语言，通过其声明式编程和动态布局特性，实现了美观且响应迅速的交互界面；核心功能如串口通信和数据处理则通过 C++ 实现，确保了通信效率和底层控制的可靠性。

主界面以动态图形形式实时展示 Delta 机械臂的当前姿态（通过关节角度计算末端位置）和目标坐标（X, Y, Z），直观反映机械臂的工作状态。操作人员可通过界面上的输入框手动输入目标坐标，精确控制机械臂末端移动至指定位置，或者通过一键执行按钮调用预设路径（如门型轨迹），实现自动化拾取和放置操作。串口通信模块通过 C++ 与 FPGA 板卡交互，支持 115200 bps 波特率，确保指令传输延迟低于 10 毫秒，满足实时控制需求。

4.3.2 语音模块实时交互

为进一步提升人机交互的便利性，本系统引入了语音模块，增强了操作人员的体验。语音模块在 Delta 机械臂开始拾取物体时，会实时播报当前抓取的物体信息，例如“抓取红色圆形物体”或“抓取蓝色六边形物体”，通过声音反馈直观地传达识别和操作状态。这一功能特别适用于需要监控多任务场景的操作人员，减少对视觉界面的依赖，从而提高了系统的实用性与人性化设计价值。

语音模块通过串口与紫光同创 PGL22G FPGA 板卡进行数据交流，接收从颜色与形状识别模块输出的物体分类结果（颜色和形状数据）以及机械臂的动作触发信号。FPGA 将这些数据封装为串口命令（波特率 115200 bps），传输至语音合成芯片，由其生成相应的语音输出。测试表明，语音模块的响应延迟约为 50 毫秒，略高于传统按钮操作的即时反馈（约 10 毫秒），主要由于串口传输和语音合成的处理开销。然而，这一延迟对实时性要求较低的监控场景影响不大，且直观性弥补了速度上的不足，显著提升了人机交互的便利性。

5 创新点

本作品在工业自动化领域通过结合多机器人协同控制、视觉感知与高精度定位技术，提出了一系列创新设计，显著提升了智能分拣系统的性能与应用价值。以下是项目的核心创新点，体现了其在技术实现和工程实践中的突破性贡献。

5.1 多机器人协同优化

本系统通过对 Delta 机械臂和平行四边形连杆机械臂的协同优化，显著提升了整体分拣效率和任务切换的稳定性，适应了工业流水线连续运行的场景需求。Delta 机械臂负责物体的拾取与分类放置，每完成三次拾取后执行复位操作，利用限位开关精确回归零点状态，有效消除因多次操作导致的累计误差。与此同时，平行四边形连杆机械臂通过磁吸方式搬运托盘，电磁铁吸力达 6 牛顿，单次搬运耗时约 5 秒，定位误差控制在 ± 2 毫米，扩展的 Y 轴运动范围（-300 至 700 毫米）确保了托盘在传送带上的灵活取放。

5.2 多级数据放大优化 FPGA 定点运算精度

为克服 FPGA 定点运算的精度限制，本设计引入了多级数据放大策略。以 Delta 机械臂逆运动学解算为例，输入坐标放大 16 倍，中间参数放大 65536 倍，输出角度放大 256 倍，所有倍数采用 2 的次方形式，便于通过移位操作高效实现。这一创新不仅避免了浮点运算的高资源消耗，还通过 ModelSim 仿真验证，Verilog 实现的解算精度与 Python 浮点计算误差仅 0.0048° ，折算末端误差小于 0.1 毫米，远超 ± 1 毫米的系统要求。这一方法在资源受限的紫光同创 PGL22G 芯片（仅 30 个 APM）上实现了高精度计算，提供了可推广的定点运算优化方案。

5.3 乘法器分时复用降低资源占用

针对 PGL22G 芯片 APM 资源的瓶颈，项目创新性地采用了乘法器分时复用技术。初期设计因使用 32 位乘法器（每个消耗 4 个 APM）导致资源超限，通过优化仅使用两个乘法器（共 8 个 APM）处理 Delta 和连杆机械臂的逆运动学解算，资源占用降至 16 个 APM，成功通过 PDS 布局布线。分时复用虽增加 20 个时钟周期延迟，但对 4.7 微秒的总解算时间影响可忽略，同时将逻辑资源利用率控制在 77%，实现了单板集成双机械臂控制的突破，显著提升了硬件效率。

5.4 逆运动学解算速度的创新优化

本作品通过在紫光同创 PGL22G FPGA 平台上实现逆运动学解算模块，充分利用 FPGA 的高并行处理能力和模块化设计，相较于传统 MCU 方案实现了显著的性能提升。传统 MCU 方案（如基于 STM32 的 C 语言实现）受限于其串行计算方式和主频（一般为 100-200 MHz），单次逆运动学解算耗时通常在 50-100 微秒之间。在 Delta 机械臂和连杆机械臂的高速连续分拣任务中，这种速度难以满足亚微秒级的实时性要求，特别是在需要频繁计算电机角度以响应动态路径规划时，延迟成为瓶颈。

通过 Verilog 实现的逆运动学解算模块，利用 FPGA 的流水线设计和并行计算能力，将解算速度大幅提升至 4.7 微秒（235 个时钟周期，50 MHz 主频），比传统 MCU 方案快 10 倍以上。

5.5 双目摄像头分区显示与实时监控

HDMI 输出模块通过创新设计，利用两个 video_rect_read_data 模块将 1024×768 显示屏划分为左右两部分，同步呈现双目摄像头的的数据。左半部分显示二值化处理结果（如边缘检测后的色块轮廓），右半部分叠加识别画框（标注颜色和形状），直观反映物体的分类和定位信息。这一分区显示设计结合 65 MHz 点时钟和 DDR3 的 Burst 读写机制，确保视频流的低延迟传输和稳定输出，单帧处理延迟控制在 50 毫秒以内。

5.6 低延迟高精度距离反馈集成

通过 I²C 接口与 VL6180X 距离传感器通信，Delta 机械臂末端实现了 0 至 10 厘米的实时测距功能，测量精度优于±1mm。设计引入了 5 次采样中值滤波算法，FPGA 缓存连续 5 次测量值并取中值，显著抑制了工业环境中振动和复杂光照（如阴影或强光反射）引起的噪声干扰。这一高精度距离反馈被无缝融入机械臂控制流程，动态调整吸盘下降高度，使拾取失败率从初始的约 10%降至 5%以下。

VL6180X 的高精度测距和快速响应与 FPGA 的高效处理能力相结合，确保了系统在动态环境下的适应性和可靠性。这一创新不仅提升了分拣任务的成功率，还为机械臂在复杂工业场景中的精准操作提供了技术保障，展现了低延迟高精度反馈的集成优势。

6 总结

本作品成功设计并实现了一个基于紫光同创 PGL22G FPGA 平台的工业分拣系统，通过 Delta 机械臂与平行四边形连杆机械臂的协同工作，完成了智能分拣任务，模拟了工业流水线场景下的多机器人低延迟、高精度协同控制需求。

6.1 功能实现

作品覆盖了赛题的基础与进阶任务，实现了完整的工业分拣流程：

- **运动控制：**Delta 机械臂通过门型路径规划实现精准拾取与放置，单次操作耗时 2 秒，定位精度 ± 1 毫米；平行四边形连杆机械臂通过磁吸搬运托盘，Y 轴范围扩展至-300 至 700 毫米，单次搬运耗时 5 秒，定位误差 ± 2 毫米。步进电机梯度加减速算法和逆运动学解算模块（Verilog 实现）确保了运动的平滑性和精确性。
- **目标识别：**利用双目摄像头（OV5640，720P）采集数据，通过 HSV 色彩空间转换识别红、黄、蓝、绿四种颜色，结合投影分析和面积阈值（圆形 6700-8000，正方形 8000-10000，六边形 4000-6700）区分三种形状（圆形、正方形、六边形）。识别耗时约 50.5 毫秒。
- **传感器集成：**通过 I²C 接口与 VL6180X 测距传感器通信，Delta 机械臂末端实现 0-10 厘米实时测距，精度 ± 1 mm，结合中值滤波降低拾取失败率至 5%以下。RFID 模块读取托盘编号，模拟物流场景。
- **人机交互：**上位机基于 Qt 框架（QML+C++）开发，支持手动输入坐标和预设路径控制，动态展示机械臂姿态。HDMI 模块以 1024 $\times$ 768@60 Hz 分辨率分区显示双目摄像头数据，结合边缘检测和画框标注，直观呈现识别结果。语音模块实时播报拾取信息，提升交互便利性。
- **多机器人协作：**Delta 机械臂每三次拾取后复位（通过限位开关），连杆机械臂同时搬运托盘，协作效率提升 20%以上，平均分拣周期缩短至 20 秒。

6.2 技术创新

作品在多个方面实现了技术创新：

- **逆运动学解算优化：**利用 FPGA 并行计算和流水线设计，解算速度提升至 4.7 微秒（235 个时钟周期，50 MHz 主频），比传统 MCU 方案（50-100 微

秒)快 10 倍以上。数据放大策略(坐标 16 倍、中间参数 65536 倍、角度 256 倍)和移位操作确保定点运算精度,误差仅 0.0048°。

- **资源优化:**通过乘法器分时复用,将 APM 占用从 123 个降至 16 个(Delta 和连杆机械臂各 8 个),在 PGL22G 芯片(30 个 APM)上实现双模块集成,逻辑资源利用率 77%,布局布线通过。
- **双目摄像头分区显示:**HDMI 模块通过两个 video_rect_read_data 模块分区显示双路视频,结合 65 MHz 点时钟和 DDR3 Burst 读写,延迟 16.7 毫秒以内,填补了传统单路显示的不足。
- **高精度距离反馈:**VL6180X 测距与 FPGA 控制集成,结合 5 次采样中值滤波,提升了复杂环境下的稳定性,拾取成功率达 95%以上。

6.3 性能表现

系统在性能测试中表现优异:

- **实时性:**逆运动学解算 4.7 微秒,满足任务需求;颜色与形状识别耗时 50.5 毫秒,需进一步优化以达到 33 毫秒目标。
- **精度:**物体定位精度 ± 1 毫米,机械臂运动精度 ± 1 毫米(Delta)和 ± 2 毫米(连杆),测距精度 ± 1 毫米。
- **稳定性:**连续运行无任务冲突或机械故障,同步带打滑问题通过速度优化解决,振动显著减少。
- **资源利用:**APM 利用率 100%,逻辑资源 77%,接近芯片极限,但功能实现稳定,验证了优化策略的有效性。

6.4 应用价值

本作品贴近工业自动化场景,成果可广泛应用于智能分拣、物流运输和柔性制造领域。双机械臂协同设计提高了流水线效率,低延迟高精度反馈适应了复杂环境,模块化设计和资源优化策略为 FPGA 在工业控制中的应用提供了参考。未来可通过引入更高性能芯片、优化识别速度及完善立体视觉算法,进一步提升系统的实时性和适应性,为工业 4.0 的智能化发展贡献力量。

7 参考文献

- [1]李浩然,李明桢,甄国涌,等.FPGA 智驱多通道步进电机精密平台[J].工业仪表与自动化装置,2025,(01):103-110.DOI:10.19950/j.cnki.CN61-1121/TH.2025.01.019.
- [2]张伟涛,全英汇,袁晓光,等.基于可重构微处理器的步进电机控制实验设计[J].电气电子教学学报,2024,46(06):195-199.
- [3]张勇,张宪民,胡俊峰,等.高速并联机械手最优时间轨迹规划及实现[J].机电工程技术,2010,39(10):42-45+108.
- [4]刘丹丹,王琚,刘杰,等.一种自适应的双臂协作码垛机器人的研发[J].机械设计与制造工程,2023,52(10):79-83.
- [5]朱训栋,徐海港,赵栋杰.码垛机械臂运动学位置正逆解分析[J].农业工程,2017,7(04):155-158.
- [6]黎显伟.码垛机器人的分类及应用[J].机械,2018,45(03):29-34+42.
- [7]梁礼群.自动化生产线搬运机械手设计研究[J].组合机床与自动化加工技术,2018,(04):162-164.DOI:10.13462/j.cnki.mmtamt.2018.04.039.
- [8]杨志杰,王平浩,郑勇.RFID 标签印刷写卡一体机机械结构设计与实现[J].物联网技术,2024,14(07):120-123.DOI:10.16667/j.issn.2095-1302.2024.07.032.
- [9]D. Austin, "Generate stepper-motor speed profiles in real time," *Embedded Systems Programming*, Jan. 2005.
- [10]H. Hadfield, L. Wei & J. Lasenby, "The Forward and Inverse Kinematics of a Delta Robot," *Advances in Computer Graphics (CGI 2020)*, Lecture Notes in Computer Science (LNIP, volume 12221), 2020, pp. 447-458.
- [11]F. Pierrot, C. Reynaud & A. Fournier, "DELTA: a simple and efficient parallel robot," *Robotica*, Volume 8, Issue 2, April 1990, pp. 105-109.
- [12]R. Andraka, "A survey of CORDIC algorithms for FPGA based computers," *Proceedings of the 1998 ACM/SIGDA Sixth International Symposium on Field Programmable Gate Arrays*, 1998, pp. 191-200.
- [13]P. Chen, M. Gao, J. Huang, Y. Yang and Y. Zeng, "High-Speed Color Sorting Algorithm Based on FPGA Implementation," 2018 IEEE 27th International Symposium on Industrial Electronics (ISIE), Cairns, QLD, Australia, 2018, pp. 235-239, doi: 10.1109/ISIE.2018.8433831.

- [14] Y. -S. Kung, K. -H. Tseng, C. -S. Chen, H. -Z. Sze and A. -P. Wang, "FPGA-Implementation of Inverse Kinematics and Servo Controller for Robot Manipulator," 2006 IEEE International Conference on Robotics and Biomimetics, Kunming, China, 2006, pp. 1163-1168, doi: 10.1109/ROBIO.2006.340093.
- [15] V. D. Cong, "Visual servoing control of 4-DOF palletizing robotic arm for vision based sorting system," International Journal on Interactive Design and Manufacturing (IJIDeM), 2023, 17, pp. 717-728.
- [16] R. Szczepanski, K. Erwinski, M. Tejer, A. Bereit & T. Tarczewski, "Optimal scheduling for palletizing task using robotic arm and artificial bee colony," Engineering Applications of Artificial Intelligence, 2022, 113, 104976.